AI and Human Narration in Child-Centered Storytelling Applications: ToggleTails

Submitted By: Noor Hasan
Supervised By: Dr. Fatema Akbar

Submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science

College of Engineering
American University of Bahrain
Riffa, Kingdom of Bahrain

December, 2025

American University of Bahrain
Approval Page

AI and Human Narration in Child-Centered Storytelling Applications: ToggleTails

By: Noor Hasan

We recommend acceptance of this manuscript in partial fulfillment of this candidate's requirements for the degree of Bachelor of Science in Computer Science. The candidate has completed the oral examination requirement of the capstone project for the degree.

Supervisor's Name	Date
Examiner's Name	Date
Examiner's Name	Date

Acknowledgments

I would like to thank my advisors, Dr. Fatema Akbar and Dr. Husain Hameed, for their guidance, patience, and encouragement throughout this project. Their support made this work possible and far more enjoyable.

To my friends, thank you for the laughter, late check-ins, and constant reminders to keep going. Your love and positive energy meant more than you know.

Contents

Glossary **VIII**

List of Figures **XII**

List of Tables **XIII**

Abstract **XIV**

1 Introduction **2**

1.1 Background and Context 3

1.1.1 Industry Context 4

1.1.2 Existing Systems 5

1.1.3 Identified Gap 5

1.1.4 Technological Trends in AI Narration 5

1.1.5 Child-Centered Design Considerations 6

1.2 Motivation 7

1.2.1 Technical Motivation 7

1.2.2 Societal Motivation 7

1.2.3 Research Motivation 7

1.3 Problem Statement 7

1.4 Societal Impact 8

1.5 Aim, Goals, and Objectives 8

1.5.1 Success Criteria 10

1.5.2 Research Questions 10

1.6 Solution Overview 11

1.6.1 System Concept 11

1.6.2 Key Features and Functionalities 11

1.6.3 Architectural Rationale 12

1.6.4 Comparison to Existing Approaches 12

1.7 Limitations and Constraints 13

1.7.1 Ethical and Privacy Considerations 13
1.8 Project Timeline 14
1.9 Summary of Chapter 1 14

2 Background and Literature Review 15
2.1 Literature Review 15
2.1.1 Evaluation of TTS Naturalness and Human-Likeness 15
2.1.2 Expressive and Controllable Speech Synthesis 16
2.1.3 AI-Generated Children Stories: Trust, Safety, and Age-Appropriateness 17
2.1.4 Interactive Storytelling Systems and Child-Facing Co-Creation . 17
2.1.5 Synthesis and Identified Gap 18
2.1.6 Exisiting Systems 19

3 Methodology and Project Plan 24
3.1 Methodological Approach 24
3.1.1 Stage 1: Problem Analysis and Requirement Definition 24
3.1.2 Stage 2: System Design 25
3.1.3 Stage 3: Prototype Implementation 25
3.1.4 Stage 4: Planned and Deferred Components 25
3.2 Methodology Overview and Summary 26

4 Requirements and Assumptions 28
4.1 Functional Requirement 29
4.2 Non-Functional Requirement 34
4.3 Design Alternatives and Rationale 37
4.4 System Overview 38
4.4.1 Introduction 38
4.4.2 System Modules, Features, and User Interactions 39
4.4.3 Overview of Workflow 40
4.4.4 Ethical Considerations and Design Trade-Offs 40

5 Design 42
5.1 Use Case Diagram 42
5.2 MVC Pattern 44
5.3 Systems Components 46
5.4 Database Design 50
5.4.1 Database Model & Storage Structure 50
5.4.2 Main Entities, Attributes, & Relationships 51

5.4.3 Constraints, Normalization, and Indexing Strategy 53

6 System Implementation 55

6.1 Introduction to the Implementation 55
6.2 Technology Stack 56
6.2.1 Mobile Application Layer 56
6.2.2 Artificial Intelligence Integration 57
6.2.3 Testing Frameworks 57
6.2.4 Development and Build Tooling 57
6.3 System Architecture and Structure 58
6.3.1 Architectural Pattern 58
6.3.2 Mobile Application Structure 59
6.3.3 Backend Architecture 60
6.4 Core Components and Features 61
6.4.1 Authentication Component 61
6.4.2 Story Generation Component 61
6.4.3 Library and Approval Filtering Component 62
6.4.4 Narration System (Native TTS and Parent Recording 63
6.4.5 Speech Recognition and Fuzzy Matching (Help Me Read) 63
6.5 API Documentation 64
6.5.1 Endpoint Organization 64
6.5.2 Authentication Module 64
6.5.3 Child Profile Management Module 65
6.5.4 Story Management and AI Generation Module 65
6.5.5 Approvals and Moderation Module 66
6.5.6 Synchronization Module 67
6.5.7 Text-to-Speech Module 67
6.5.8 Analytics Module 68
6.5.9 Request and Response Structure 68
6.5.10 Middleware and Error Handling 68
6.6 Error Handling and Logging 69
6.7 Security Considerations 69
6.7.1 JWT-Based Authentication Mechanism 69
6.7.2 Password and PIN Hashing 70
6.7.3 Enviroment Variable Management 71
6.7.4 Route-Level Access Control 71
6.8 Parent PIN Storage Strategy 71
6.8.1 Identified Security Limitations 72

6.8.2 Summary of Security Implications 72
6.9 Dependencies and Package Management 73
6.9.1 Backend Dependencies 73
6.9.2 Frontend Dependencies 73
6.9.3 Frontend Stack 73
6.9.4 Database and AI Integration 74
6.9.5 Build and Deployment Tooling 74
6.9.6 Expo Application Services (EAS) 75
6.9.7 Environment Configuration 75
6.9.8 Continuous Integration and Deployment Status 76

7 Testing 77
7.1 Introduction to Testing 77
7.2 Types of Testing Conducted 77
7.3 Design and Implementation Challenges 78
7.4 Functional Testing 79
7.5 Objective-Linked Testing 80
7.6 Usability Testing and User Acceptance 82
7.6.1 Child Observation Results 82
7.6.2 Parent Observation Results 83
7.7 Limitations of Testing 85

8 Conclusion 86
8.1 Conclusion Future work 86

References 88

Glossary

Artificial Intelligence (AI)
A branch of computer science focused on developing systems capable of performing tasks that normally require human intelligence, such as speech processing, learning, and decision-making.

Text-to-Speech (TTS)
A technology that converts written text into spoken audio using synthetic or AI-generated voices.

Neural TTS
A modern approach to TTS using deep learning models to generate highly natural-sounding synthetic speech.

Prosody
The rhythm, stress, and intonation patterns in speech that influence its expressiveness and naturalness.

Phoneme
The smallest unit of sound in speech. TTS systems use phonemes to control pronunciation and speech rhythm.

Voice Modeling
The process of using AI to replicate a speaker's voice characteristics from audio samples.

AI-Generated Narration
Narration produced using artificial intelligence techniques, specifically text-to-speech systems, to transform written story content into spoken audio.

Human Narration
Story narration delivered using pre-recorded audio spoken by a human narrator, such as a parent or guardian.

Dual-Mode Narration
A system capability that supports both human-recorded narration and AI-generated narration within the same storytelling application.

Parent View
A protected interface intended for parents or guardians that allows configuration of child profiles, narration settings, content approval, and access restrictions.

Child View
A simplified and age-appropriate interface designed for child users, providing access only to approved stories and narration playback features.

Role-Based Access Control (RBAC)
A security mechanism that restricts system access based on user roles, ensuring that children cannot access administrative or configuration features.

Offline Capability
The ability of a system to function without an active internet connection, ensuring reliability, accessibility, and privacy-preserving operation.

Local-First Architecture
A system design approach in which data is stored and processed primarily on the user's device rather than relying on continuous cloud connectivity.

Model–View–Controller (MVC)
A software architectural pattern that separates an application into three components: the Model for data and logic, the View for user interface, and the Controller for handling interactions.

Narration Pipeline
The structured process through which story text is prepared, converted into audio, and delivered for playback within the application.

Story Library
A curated collection of approved stories available to the child, presented in a visual and accessible format.

Privacy-by-Design
A system design principle that embeds privacy protection measures into the system architecture from the outset, minimizing unnecessary data collection and exposure.

Multimodal Interface
An interface combining various input and output methods—such as voice, text, and visuals—for a richer user interaction experience.

MOS (Mean Opinion Score)
A widely used metric for evaluating speech quality based on listener ratings, typically on a scale from 1 to 5.

Distributional Evaluation
A method that evaluates how closely the statistical distribution of generated speech aligns with natural human speech patterns.

Human Fooling Rate
The percentage of times users are unable to distinguish between synthetic and human speech, used to evaluate TTS realism.

LLM (Large Language Model)
A deep learning model trained on massive text data to generate coherent and contextually relevant language outputs.

ASR (Automatic Speech Recognition)
A technology that converts spoken audio into written text, commonly used in voice-based educational or read-along systems.

Age-Appropriate Language
Language and sentence structure tailored to be understandable and engaging for specific age groups, especially children.

Trustworthiness (in AI)
The degree to which AI outputs can be relied upon as accurate, safe, and ethically appropriate for the user context.

ToggleTails
The proposed child-centered storytelling application developed in this project to support controlled comparison between human and AI narration.

List of Figures

5.1 Use Case Diagram for the ToggleTails Prototype 43
5.2 MVC-Inspired Layered Architecture of ToggleTails 46
5.3 Entity Relationship Diagram for Local Storage in ToggleTails 50
5.4 Major System Components and Their Interactions in ToggleTail 53

6.1 Code Sample 70
6.2 Mobile App Architecture and Technologies 74

List of Tables

1.1 Project Timeline and Key Deliverables 14

2.1 Overview of Existing Storytelling and Narration Systems 19

5.1 System Components Mapped to Functional Requirements 49

6.1 Technology Stack Overview 58

7.1 Implementation Challenges Identified During Development 79
7.2 Functional Testing Results Linked to ToggleTail Project Objectives . . . 80
7.3 Objective-Linked Evaluation of ToggleTail System Features 81

Abstract

Hasan, Noor, “AI and Human Narration in Child-Centered Storytelling Applications: ToggleTails”, Bachelor of Science in Computer Science, December 2025, Dr. Fatema Akbar.

This capstone project presents the design and implementation of ToggleTails, a child centered storytelling application that integrates both artificial intelligence–generated narration and human-recorded narration within a unified, privacy-aware system architecture. Recent advancements in text-to-speech (TTS) and voice modeling technologies have accelerated the adoption of AI narration in educational and entertainment platforms. This project addresses these limitations by engineering a dual-narration system that allows controlled switching between AI and human narration modes.

The primary contribution of this work is technical rather than experimental. The project follows a design-and-build engineering methodology focused on system architecture, feasibility, and implementation validation. A modular mobile architecture is proposed and implemented to support parent- and child-specific interfaces with role-based access control, local data storage, in addition to flexible narration pipelines. Guardians would be able to configure story access, narration preferences, and usage limits, whereas children interact with a simplified and intuitive playback interface. A signature feature of the system is personalized narration, which enables guardians to record their own voice and use it directly or as input for AI-assisted narration, a capability not commonly supported in existing platforms.

Although storytelling provides the application context, ToggleTails establishes a foundation for future enhancements such as interactive reading feedback, comprehension prompts, and small-scale observational studies. Combining AI and human narration in child-focused storytelling applications, positioning AI as a supportive tool rather than a replacement for human interaction.

“‘latex

Chapter 1

Introduction

Advances in artificial intelligence have fundamentally expanded the possibilities of expressive text-to-speech (TTS) systems, enabling the generation of natural, human-like narration at scale. These technologies have gained significant traction across intelligent multimedia systems, assistive interfaces, digital education platforms, and automated content delivery. Despite this progress, commercially available narration systems remain largely generic in design — optimized for broad audiences rather than for contexts requiring personalized, configurable, and privacy-preserving audio experiences. In particular, no widely adopted system provides a unified architecture capable of integrating parent-recorded human narration alongside AI-generated speech within a single, modular playback pipeline.

The deployment of narration technologies within child-focused interactive applications introduces a distinct set of engineering constraints: secure handling of locally stored data, reliable offline operation, strict role-based access control, and the need for interfaces calibrated to the cognitive and attentional characteristics of young users. Addressing these constraints within a cohesive system architecture represents a meaningful and underexplored engineering challenge.

This project addresses that challenge through the design and implementation of *ToggleTails*, a child-centered mobile storytelling application that delivers dual-mode narration — combining AI-generated TTS audio with parent-recorded voice playback — within a unified, privacy-aware local-first architecture. A defining contribution of the application is its parent narration system: guardians may record their own voice for individual stories, and those recordings are stored locally and played back automatically when a child opens the corresponding story. Following a successful recording save, the system additionally submits the audio to the ElevenLabs voice cloning API; if cloning succeeds, the resulting voice model is stored and used for AI-synthesized narration of that story, combining the familiarity of the parent's voice with the flexibility of AI-generated playback. The application further includes a curated built-in library of public

domain stories, AI-assisted TTS narration via ElevenLabs with native on-device fallback, and an interactive read-along mode that supports speech recognition for guided reading assistance.
Although storytelling provides the practical application context, the primary contribution of this project lies in the design and engineering of the underlying narration architecture. Lightweight observational testing with child and parent participants served to validate system behavior and inform iterative design refinement. These observations are not intended as formal experiments or structured research studies, but rather as implementation validation activities that ensure the system performs correctly in realistic use conditions. The project situates itself firmly within the discipline of designing custom, secure, and AI-integrated narration systems for child-facing mobile applications.

1.1 Background and Context

Project Domain in Computer Science

This project draws upon multiple established sub-fields of Computer Science to produce an adaptive, interactive, and privacy-preserving storytelling platform. It integrates principles from intelligent multimedia systems to manage dynamic narration workflows, applies human-computer interaction design methodologies to construct age-appropriate and role-differentiated interfaces, and leverages mobile systems architecture to ensure reliable offline functionality, secure local storage, and modular component separation.

- **Intelligent Multimedia Systems**, involving dynamic narration pipeline design, multi-modal audio processing, and coordinated playback across human-recorded and AI-synthesized audio sources.
- **Human–Computer Interaction (HCI)**, through the design and implementation of distinct child-appropriate and parent-controlled interfaces, each calibrated to their respective users' cognitive and operational needs.
- **Artificial Intelligence**, specifically neural text-to-speech synthesis via the ElevenLabs API with native on-device TTS as a resilient fallback, ElevenLabs voice cloning applied as an enhancement layer following parent recording, and GPT-4o-mini for AI-powered story generation.
- **Mobile Systems Architecture**, emphasizing offline-first reliability, secure local storage via AsyncStorage and SecureStore, modular domain-layered design, and efficient data management without dependency on continuous cloud connectivity.

Significance and Contribution

The system provides three primary technical contributions:

1. A modular dual-mode narration pipeline capable of integrating parent-recorded human narration and AI-generated TTS audio within a unified playback architecture, with automatic source resolution based on availability and parent configuration.

2. A parent narration recording system that enables guardians to record their own voice for individual stories on a page-by-page basis. Recordings are persisted locally and played back automatically when the child selects the corresponding story. Following a successful save, the system additionally submits the recording to the ElevenLabs voice cloning API as an enhancement layer; if cloning succeeds, the resulting voice model is stored and used for AI-synthesized narration of that story going forward, combining the familiarity of the parent's voice with the flexibility of AI-generated playback.

3. A customizable, child-oriented application architecture that provides strong privacy controls, full offline functionality, a built-in curated story library, role-based access control, and flexible configuration options not available in existing consumer storytelling platforms.

Scope of the Chapter

This chapter introduces the project context, identifies the technical motivation and problem being addressed, defines the project aims and objectives, summarizes the proposed solution and its architectural rationale, clarifies constraints and ethical considerations, and outlines the structure of the full report.

1.1.1 Industry Context

The adoption of AI-driven narration has grown substantially as TTS systems continue to improve in expressiveness, prosodic naturalness, multilingual capability, and voice quality. These technologies are now actively integrated into audiobooks, digital learning platforms, voice assistants, and accessibility tools at scale. However, the majority of commercially deployed narration systems remain cloud-dependent, constrained by fixed and non-configurable voice libraries, and optimized for general audiences rather than for personalized or context-specific storytelling experiences. The integration of

human-recorded audio alongside AI synthesis, particularly within child-facing applications requiring offline operation and parental oversight, remains an area that existing platforms have not meaningfully addressed.

1.1.2 Existing Systems

Established platforms such as Amazon Alexa Stories and Audible's AI Narration deliver high-quality synthetic audio but operate within closed, proprietary ecosystems that do not permit user-generated voices, configurable narration modes, or parent-controlled access management. These systems provide no mechanism for guardians to introduce their own recorded narration, nor do they support offline content delivery, role-differentiated interfaces, or controlled comparison between human and AI audio sources. As a result, they are structurally unsuited to the requirements of a child-centered storytelling environment in which parental agency, privacy, and offline reliability are design priorities.

1.1.3 Identified Gap

Existing systems are limited in several ways that directly motivate the development of this application:

- They do not support controlled narration comparison across human-recorded and AI-generated audio modes within a single application.
- They do not provide mechanisms for parents or guardians to record and integrate their own narration into the child's reading experience.
- They lack modular architectures suitable for offline operation, user-driven narration configuration, and secure local data management.
- They do not offer simple, secure environments purpose-built for young users, with appropriate interface constraints and access separation.
- They provide no built-in support for lightweight system validation activities or behavioral observation of narration use in child contexts.

1.1.4 Technological Trends in AI Narration

Recent advances in neural text-to-speech synthesis, prosody modeling, and automatic speech recognition have substantially raised the quality ceiling for AI-driven narration.

Modern TTS systems such as ElevenLabs are capable of generating expressive, emotionally varied speech with fine-grained control over rhythm, stress, and intonation — qualities essential for engaging children's storytelling. Voice cloning technology has further expanded these possibilities, enabling systems to synthesize speech that closely replicates the acoustic characteristics of a specific individual from a short audio sample. Simultaneously, automatic speech recognition has become increasingly accessible on mobile platforms, enabling interactive read-along features in which a system can listen to a child reading aloud and provide real-time feedback.

Bringing these capabilities into a mobile environment introduces non-trivial engineering challenges. Mobile devices operate under constraints of limited processing power, fixed storage capacity, and strict data protection requirements — particularly when the target user group involves minors. These constraints necessitate an architecture that relies primarily on on-device processing and locally cached assets, supplementing with cloud-based AI services only where necessary and under clearly defined conditions. The design decisions throughout this project are strongly shaped by these considerations.

1.1.5 Child-Centered Design Considerations

Applications designed for young users must adhere to principles of minimal cognitive load, predictable interaction patterns, and visual simplicity. Interfaces must be navigable with minimal depth, use large and clearly labeled controls, and avoid exposing administrative complexity to child users. Offline operation is a functional requirement rather than an optional enhancement, as children may use the application in home environments without consistent connectivity. Strong privacy protections are essential: all child-related data, including usage patterns and any audio interaction, must be handled locally and never transmitted without explicit guardian authorization.

Meeting these requirements imposes concrete engineering constraints on the system. Access control must be enforced at the architectural level, ensuring that children cannot access parent configuration areas. State persistence must be reliable across sessions even in offline conditions. The narration pipeline must handle source switching gracefully and without perceptible interruption. Together, these constraints shape the layered architecture adopted by this project and determine which technologies and design patterns are appropriate for a child-facing production application.

1.2 Motivation

1.2.1 Technical Motivation

This project is technically motivated by the need to engineer narration pipelines capable of managing multiple heterogeneous audio sources — specifically, AI-synthesized TTS audio, ElevenLabs voice-cloned narration, and human-recorded playback — within a single unified system. The coexistence of these modes within one architecture introduces non-trivial design challenges around source resolution, fallback behavior, local storage management, and interface consistency. Building and validating such a system provides a technically meaningful contribution independent of the storytelling context in which it operates.

1.2.2 Societal Motivation

Although the project's primary contribution is technical, it aligns with broader goals of responsible AI integration in educational and child-facing contexts. By placing complete narration control in the hands of parents and guardians — including the choice between AI and human voices, story access permissions, and usage limits — the system positions AI as a configurable and accountable tool rather than an autonomous one. The application's emphasis on privacy-by-design and offline-first operation reflects and responds to growing societal expectations around secure, transparent, and parent-supervised digital environments for children.

1.2.3 Research Motivation

The system is designed to support small-scale, non-formal observational activities with child and parent participants. These activities provide insight into how narration modes perform in realistic use conditions, how children interact with the playback interface, and how parents engage with configuration and recording workflows. While not constituting structured research, these observations serve as a validation mechanism that informs iterative design improvement and supports the practical credibility of the system as a functional prototype.

1.3 Problem Statement

Despite the growing availability and quality of AI narration tools, no existing platform provides a modular architecture capable of integrating parent-recorded human nar-

ration alongside AI-generated TTS audio within a single, offline-capable, child-safe application. Existing systems lack support for parent-driven narration configuration, voice cloning initiated by guardians, role-differentiated interfaces, and structured narration mode switching. This project addresses the engineering challenge of designing and implementing a dual-mode narration system that overcomes these structural limitations while meeting the privacy, usability, and access control requirements of a child-centered mobile application.

1.4 Societal Impact

Beyond its technical scope, this project reflects a broader societal reality in which artificial intelligence is increasingly embedded across education, entertainment, communication, and caregiving. As AI systems become commonplace in spaces traditionally shaped by human interaction — including children's storytelling and early literacy environments — it becomes critically important to understand not only how these technologies function, but how they are experienced and interpreted by the people who use them. This project situates AI narration within that wider context, recognizing that AI is not confined to specialized technical domains but is instead becoming a pervasive presence in everyday domestic and educational life. By embedding AI within a controlled, parent-supervised storytelling application, the project demonstrates that expressive AI narration can be deployed responsibly, transparently, and in service of child development rather than in place of human engagement.

1.5 Aim, Goals, and Objectives

Aim:
To design and implement a customizable, privacy-preserving mobile storytelling application that supports dual-mode narration — combining AI-generated text-to-speech audio and ElevenLabs voice-cloned narration with parent-recorded voice playback — within a modular, offline-capable architecture, while providing parent-controlled content management, role-differentiated interfaces, and a built-in curated story library accessible without external connectivity.
Goals:

1. Develop a fully functional mobile prototype that delivers high-quality story narration through AI-generated TTS audio, ElevenLabs voice cloning, and direct parent-recorded playback, selectable and configurable by guardians.

2. Enable parents or guardians to record their own narration for individual stories, with recordings stored locally, played back automatically, and optionally submitted for ElevenLabs voice cloning to produce a personalized AI narration model.

3. Enable parents or guardians to customize narration mode, story permissions, and usage limits through a secure, PIN-protected configuration interface.

4. Provide children with a simplified, age-appropriate playback interface offering multiple reading modes, including passive listening, self-directed reading, and guided read-along with speech recognition assistance.

5. Ensure privacy-preserving, offline-first operation with all data handled through secure local storage and robust role-based access control.

6. Create a modular system architecture that supports long-term maintainability and scalability for future enhancements to narration quality, interactivity, and content delivery.

Objectives:

- Implement a layered mobile architecture separating presentation, domain logic, and data management to ensure clear separation of concerns and long-term maintainability.

- Develop a narration pipeline capable of delivering AI-generated TTS audio via ElevenLabs with native on-device TTS as a resilient fallback, playing back parent-recorded narration, and applying ElevenLabs voice cloning as an enhancement layer following successful recording saves.

- Build a secure parent interface with bcrypt-hashed password authentication and SecureStore encryption to manage story approval, narration preferences, recording workflows, and child usage settings.

- Design a child-friendly playback interface supporting Read Myself, Listen to Story, and Read with Help modes with simple navigation and intuitive controls.

- Bundle a curated library of fifty-two public domain stories across six genre categories, seeded automatically into local storage on first launch with no internet dependency.

- Integrate secure local storage to persist narration recordings, story content, approval state, and personalization settings entirely on-device.

- Provide an AI story generation capability via GPT-4o-mini, accessible through the parent interface, that produces personalized stories based on child profile parameters including age, reading level, and interests, operational when the backend server is running and degrading gracefully when unavailable.
- Define an extensible architecture capable of incorporating future enhancements, including expanded speech recognition features, comprehension prompts, and adaptive narration based on child reading progress.
- Prepare the prototype for lightweight observational validation activities that inform iterative system refinement without embedding formal data collection mechanisms into the application itself.

1.5.1 Success Criteria

The success of this project is evaluated against the stability, usability, and technical reliability of the implemented system. A successful prototype delivers uninterrupted narration playback across all supported modes — AI-generated TTS, ElevenLabs voice-cloned narration, and parent-recorded audio — with graceful fallback between sources when a preferred mode is unavailable. The application must operate fully offline, with all story content, narration recordings, and configuration data persisted securely through local storage. The child interface must remain accessible and navigable without adult assistance, while the parent interface must enforce access control reliably and support all configuration workflows without error. Finally, narration mode switching must occur without perceptible performance degradation, ensuring a consistent and responsive experience across the full application.

1.5.2 Research Questions

Although the primary contribution of this project is engineering rather than empirical research, an auxiliary question guides the observational validation component of the work: *How can a dual-mode narration architecture be engineered to accommodate human-recorded, AI-generated, and voice-cloned audio within a single unified playback pipeline, and what design considerations emerge from lightweight observational use with child participants?* This question does not constitute a formal research hypothesis, but rather provides conceptual grounding for the system validation activities and the iterative design refinements they inform. It remains secondary to the engineering focus of the project while establishing a foundation for potential future investigation into the comparative effects of narration mode on child engagement and comprehension.

1.6 Solution Overview

1.6.1 System Concept

ToggleTails is a child-centered mobile storytelling application built on a local-first architecture that requires no continuous internet connectivity. The system integrates human-recorded and AI-generated narration within a unified playback pipeline, provides parent-controlled configuration through a bcrypt-authenticated interface, and delivers story content to children through a simplified, age-appropriate reading environment. All data — including narration recordings, story content, approval state, and usage settings — is stored securely on-device using structured local storage.

1.6.2 Key Features and Functionalities

- **Dual Narration Modes:** Parent-recorded human narration and AI-generated TTS narration via ElevenLabs, with native on-device TTS as a resilient fallback. Following a successful parent recording save, the system additionally attempts ElevenLabs voice cloning to produce a personalized AI narration model for that story.

- **Built-in Story Library:** Fifty-two curated public domain stories across six genre categories — Animals, Adventure, Bedtime, Science, Values, and Fantasy — bundled with the application and seeded into local storage at first launch with no internet dependency.

- **Parent Recording Workflow:** Guardians may record their own narration for any story on a page-by-page basis. Recordings are stored locally and activated automatically when the child selects the corresponding story.

- **Multiple Reading Modes:** Children may choose between Read Myself (self-paced), Listen to Story (passive narration), and Read with Help (guided read-along with speech recognition assistance, active in native builds).

- **Parent Configuration Interface:** PIN-protected access for managing story approval, narration mode preference, voice label settings, and usage time limits. The interface also supports AI-powered story generation via GPT-4o-mini, which produces personalized stories based on child profile parameters including age, reading level, and interests. This feature is operational when the backend server is running and reachable, and degrades gracefully with an appropriate error state when the server is unavailable.

- **Offline-First Operation:** All core functionality — story browsing, narration playback, and configuration — operates without internet connectivity. ElevenLabs audio generation and voice cloning require connectivity during processing; generated audio and cloned voice models are subsequently cached locally for offline access.
- **Event Logging:** Reading sessions, story completions, and narration mode selections are logged locally to support usage monitoring and iterative design refinement.

1.6.3 Architectural Rationale

A three-layer architecture — separating the presentation layer, domain logic layer, and data management layer — ensures scalability, maintainability, and clear separation of concerns. Offline-first local storage via AsyncStorage and SecureStore provides privacy and reliability without dependency on external infrastructure. The narration pipeline is implemented as a modular service layer that resolves the appropriate audio source at runtime based on availability and parent configuration, enabling seamless switching between narration modes without disruption to the child interface. Story content is bundled as static assets and seeded automatically on first launch, eliminating the need for network access to populate the library.

1.6.4 Comparison to Existing Approaches

Unlike commercial storytelling platforms, ToggleTails offers:

- Parent-driven narration recording with direct playback and optional ElevenLabs voice cloning as an enhancement layer.
- Configurable dual-mode narration with runtime source resolution and automatic fallback.
- A unified human and AI playback architecture within a single modular pipeline.
- Complete offline functionality and privacy-by-design with no mandatory cloud dependency.
- A built-in curated story library requiring no external content provisioning or third-party integration.
- A platform structurally suited to lightweight observational comparison of narration modes with child users.

1.7 Limitations and Constraints

The scope of this project is centered on the engineering and validation of the narration system and application architecture rather than large-scale empirical evaluation. Story content within the application consists of curated public domain literature bundled with the application, as well as AI-generated stories produced through an integrated story generation feature accessible to parents. The application does not engage in autonomous story generation without explicit parent initiation.

The guided read-along mode (Read with Help), which utilizes automatic speech recognition, requires a native application build via Expo Application Services and is not active within the Expo Go development environment. This is a known platform constraint and is handled gracefully within the application through informative messaging. All other narration modes function fully within both development and production environments.

Multi-device synchronization and cloud-based data persistence are excluded from this phase of development to preserve offline functionality and maintain the privacy-by-design architecture. AI story generation via GPT-4o-mini and ElevenLabs voice cloning require active API keys and server connectivity during processing; in the absence of these, the system falls back gracefully to ensure uninterrupted core functionality.

1.7.1 Ethical and Privacy Considerations

All data used by the application is stored locally on the device. No personal information, usage data, or narration recordings are transmitted to external servers beyond the explicit AI processing requests initiated by the parent — specifically, ElevenLabs voice cloning and GPT-4o-mini story generation — which are performed only on guardian action and are not automated. Role-based access control enforced at the architectural level ensures that children cannot access or modify parent configuration areas, narration settings, or story management features. Voice recordings and story materials are stored within the application's protected local directories using SecureStore encryption, and the system is designed to minimize data collection to only what is necessary for its core functions. This approach upholds strong privacy standards, reduces exposure to security risk, and reflects the ethical obligations inherent in designing software for use by minors.

1.8 Project Timeline

Phase	Duration	Key Deliverables
Phase I – Prototype Design and Development	Oct – Dec 2025	Implementation of a local-first MVP prototype demonstrating core system architecture, parent control interface, child playback interface, dual narration pipeline, and built-in story library.
Phase II – Prototype Validation and Design Review	Jan – Feb 2026	Validation of architectural decisions, narration pipeline behavior, and interface usability through structured testing and lightweight observational activities with child and parent participants.
Phase III – Planned System Enhancement	Mar – Apr 2026	Design-level and implementation-level refinements based on validation findings, including improvements to narration quality, interface usability, story library coverage, and system robustness.
Final Report and Presentation	May 2026	Final documentation, system evaluation summary, and formal presentation of design decisions, prototype outcomes, and future development roadmap.

Table 1.1: Project Timeline and Key Deliverables

1.9 Summary of Chapter 1

Chapter 1 established the technical context and engineering motivation underlying the ToggleTails project. It identified the gap in existing narration platforms that this work addresses, defined the project aim, goals, and objectives with reference to what the implemented system delivers, introduced the solution concept and its architectural rationale, and clarified the scope, limitations, and ethical considerations that constrain the prototype. The chapter also outlined the project timeline and set the groundwork for the detailed requirements specification, architectural design, and system implementation discussions that follow in subsequent chapters.

Chapter 2

Background and Literature Review

2.1 Literature Review

Recent progress in text-to-speech (TTS) and generative AI has made it feasible to deliver story narration at scale, but the literature shows that matching human narration quality aswell as trust is still not fully solved. This review synthesizes work across first the evaluation of speech naturalness and human-likeness, second the controllable and expressive speech synthesis. And last AI-supported children storytelling systems and safety/trustworthiness issues, in order to identify gaps that motivate the proposed direction.

2.1.1 Evaluation of TTS Naturalness and Human-Likeness

A recurring problem in TTS research is that objective like metrics frequently do not align well with human judgement, especially when models become strong enough that differences are subtle. SpeechJudge proposes a large-scale human preference dataset (paired comparisons) and a benchmark for speech naturalness judgement, then trains a reward model to better match human perception of naturalness [1]. The reported gaps between automated judgement and human agreement suggest that evaluation remains a limiting factor for reliable iteration, not only the model itself [1].
Complementary to human-preference datasets, EmergentTTS-Eval introduces stress-test style evaluation covering multiple challenging scenarios which include: emotion, paralinguistics, and complex pronunciation. All as the usage of a model-as-a-judge evaluation with large audio language models [2]. This essentiallly benchmark highlights that performance differences appear when prompts become linguistically or prosodically complex, which is common in expressive storytelling [2].
TTSDS2 further argues and suggests for distributional evaluation: which would mean

instead of isolated signals, it evaluates how closely generated speech matches real speech across domains and languages, and reports consistent correlation with subjective scores across domains [3]. Together, these works imply that "high MOS" alone can be misleading because evaluation protocols and coverage of stress cases can change the conclusion [1, 2, 3].

Beyond judgement models and metrics, a Turing-like framing has been used to quantify whether listeners confuse synthetic speech for human speech. Introducing a new terminology known as "Human Fooling Rate" is proposed as a direct deception-style metric for modern TTS, reporting that even strong systems may not consistently pass a human deception test [4]. This reinforces that for story narration, perceived "realness" and sustained listening comfort still matters and should be measured, not assumed [4].

2.1.2 Expressive and Controllable Speech Synthesis

As for the Expressive narration it depends on prosody control, emotion control, and voice quality control. A systematic review by Barakat et al. surveys deep-learning approaches for expressive speech synthesis and summarizes recurring challenges, which include and are not limited to expressive datasets, style leakage/entanglement, and evaluation difficulties [5]. These challenges become more visible and clear specifically in storytelling because the text contains frequent emotional shifts and long-range structure [5].

Voice quality is another layer beyond emotion and prosody. Rautenberg et al. study manipulation of perceptual voice qualities which include breathiness and roughness. Using a conditional normalizing flow thats then further integrated into TTS, which avoids direct acoustic correlate editing. And instead rely entirely on learned transformations [8]. Their expert evaluations show that some qualities (like breathiness) are easier to manipulate than others (like roughness), and strong edits can reduce speaker similarity [8]. For story narration, this indicates a tradeoff: increasing expressiveness may degrade perceived identity consistency, which can distract listeners or reduce trust in the narrator voice [8].

A more dataset-driven direction is StoryTTS, which provides a storytelling-style dataset with rich expressiveness annotations, emphasizing that textual expressiveness (rhetorical and linguistic cues) can be leveraged to generate more expressive speech [9]. This suggests that narration quality is not only the TTS model; it also depends on how story text is annotated/structured and how expressive cues are represented during generation [9].

2.1.3 AI-Generated Children Stories: Trust, Safety, and Age-Appropriateness

When narration is driven by AI-generated stories, content quality and safety directly affects the user experience. Bhandari and Brennan evaluate the trustworthiness of LLM-generated children's stories and conclude that models can fail to match the nuance and quality of real stories, and may include problematic content, meaning safety cannot be assumed [11]. Valentini et al. examine automatic generation and simplification of children's stories, finding that current LLMs struggle to reliably constrain vocabulary/readability for younger ages, and simplification models may require domain-specific tuning to work well for child-directed text [12]. These findings imply that for child audiences, both language level and safety filtering must be considered as first-class design constraints, not an afterthought [11, 12].

2.1.4 Interactive Storytelling Systems and Child-Facing Co-Creation

Beyond passive listening, interactive and multimodal storytelling systems continue to attempt to engage children in narrative construction. "Colin" (a multimodal human-AI co-creation system) supports children's narrative skills through voice and visual modalities and reports improvements in engagement and narrative skill measures in a user study [13].

Tinker Tales proposes a tangible board-game-like framework integrating speech interaction and AI assistance to support narrative development and AI literacy [14]. Toyteller explores toy-playing and symbol manipulation as a means to steer story generation and produce visual outputs that accompany story text [15]. DrawTalking shows sketching-and-speaking interfaces to construct interactive worlds, emphasizing user control without code [16]. While these systems show that multimodal interaction can raise engagement, many do not focus on controlled comparison of narration voices or on the speech naturalness evaluation needed for long-form listening [13, 14, 15, 16].

2.1.5 Synthesis and Identified Gap

Across the surveyed literature, three gaps appear repeatedly. First, TTS evaluation is improving (human-preference datasets, distributional metrics, stress tests), but there is still no single accepted evaluation that captures sustained, story-like listening with expressive variation [1, 2, 3, 4]. Second, controllable expressiveness (prosody, emotion, and voice quality) is advancing, yet stronger controls can introduce entanglement or reduce speaker identity consistency, which may harm the narration experience [6, 7, 8, 5]. Third, for children, story generation introduces additional constraints: age-appropriate language, safety, and trustworthiness are not reliably guaranteed by LLMs today, and must be actively managed [11, 12].

Therefore, the literature motivates a child-centered narration setting where (i) narration quality is evaluated with human-aligned protocols, (ii) expressive control is used carefully to avoid unnatural artifacts, and (iii) story content is constrained for age-appropriateness and trust. This combined framing is still not consistently addressed in prior systems, and it remains a open practical gap in current work.

2.1.6 Exisiting Systems

Table 2.1: Overview of Existing Storytelling and Narration Systems

System	Key Features	AI Integration	Limitations	Reference
Google Read Along	Interactive reading aid with real-time speech feedback	Automatic speech recognition (ASR) for read-along interaction	Not designed for narration comparison or voice customization	[17]
NovelAI	Story generation with user-defined prompts	Large language models (LLMs) for narrative text generation	Focused on text generation; no audio narration or TTS support	[18]
Audible AI Narration	Scalable audiobook narration pipeline	Neural TTS for large-scale voice generation	Limited evaluation of child comprehension or personalization	[19]
Talkify	Web-based narration platform with selectable voices	Standard TTS engines with multilingual support	Generic voices; lacks educational focus or customization	[20]
Story Speaker (Google)	Story authoring and playback via Google Assistant	Google Assistant TTS integration	Template-based; limited adaptation for children interaction	[21]
SAGA	AI-assisted story construction through guided prompts	LLM-based suggestions during story creation	Optimized for creation, not passive listening or narration quality	[22]

As shown in Table 2.1, current AI-based storytelling systems vary significantly in their approach, capabilities, and intended audiences. While platforms such as Google Read Along and Story Speaker focus on interactive reading support using speech technologies [17, 21], they do not provide mechanisms for evaluating narration quality or comparing different narration styles. Other systems, including Audible's neural TTS pipeline and Talkify, emphasize scalable voice synthesis and accessibility, but prioritize production efficiency over educational adaptation and personalization [19, 20].

Creative tools such as NovelAI and SAGA support AI-assisted story generation, yet they are primarily designed for content creation rather than passive listening or comprehension assessment [18, 22]. Overall, these limitations reveal a gap in existing systems: none explicitly support dual narration modes (human and AI), controlled voice personalization, and child-centered design within a single, privacy-aware storytelling framework.

References

[1] X. Zhang, C. Wang, H. Liao, Z. Li, Y. Wang, L. Wang, D. Jia, Y. Chen, X. Li, Z. Chen, and Z. Wu, "SpeechJudge: Towards Human-Level Judgment for Speech Naturalness," *arXiv preprint arXiv:2511.07931*, 2025.

[2] R. R. Manku, Y. Tang, X. Shi, M. Li, and A. Smola, "EmergentTTS-Eval: Evaluating TTS Models on Complex Prosodic, Expressiveness, and Linguistic Challenges Using Model-as-a-Judge," *arXiv preprint arXiv:2505.23009*, 2025.

[3] C. Minixhofer, O. Klejch, and P. Bell, "TTSDS2: Resources and Benchmark for Evaluating Human-Quality Text to Speech Systems," *arXiv preprint arXiv:2506.19441*, 2025.

[4] P. S. Varadhan, S. Thomas, S. T. M. S, S. Bhooshan, and M. M. Khapra, "The State Of TTS: A Case Study with Human Fooling Rates," in *Proc. Interspeech*, 2025, pp. 2285–2289, doi: 10.21437/Interspeech.2025-2765.

[5] H. Barakat, O. Turk, and C. Demiroglu, "Deep learning-based expressive speech synthesis: a systematic review of approaches, challenges, and resources," *EURASIP Journal on Audio, Speech, and Music Processing*, vol. 2024, no. 1, 2024, doi: 10.1186/s13636-024-00329-7.

[6] N. Ellinas, M. Christidou, A. Vioni, J. S. Sung, A. Chalamandaris, P. Tsiakoulis, and P. Mastorocostas, "Controllable speech synthesis by learning discrete phoneme-level prosodic representations," *Speech Communication*, vol. 146, pp. 22–31, 2023, doi: 10.1016/j.specom.2022.11.006.

[7] K. Zhou, Y. Zhang, S. Zhao, H. Wang, Z. Pan, D. Ng, C. Zhang, C. Ni, Y. Ma, T. H. Nguyen, J. Q. Yip, and B. Ma, "Emotional Dimension Control in Language Model-Based Text-to-Speech: Spanning a Broad Spectrum of Human Emotions," *arXiv preprint arXiv:2409.16681*, 2024.

[8] F. Rautenberg, M. Kuhlmann, F. Seebauer, J. Wiechmann, P. Wagner, and R. Haeb-Umbach, "Speech Synthesis along Perceptual Voice Quality Dimensions," *arXiv preprint arXiv:2501.08791*, 2025.

[9] S. Liu, Y. Guo, X. Chen, and K. Yu, "StoryTTS: A Highly Expressive Text-to-Speech Dataset with Rich Textual Expressiveness Annotations," *arXiv preprint arXiv:2404.14946*, 2024.

[10] M. Daniluk and A. P. Pietrzak, "Comparative analysis of natural and synthesized Polish speech," *International Journal of Electronics and Telecommunications*, pp. 361–366, 2024, doi: 10.24425/ijet.2024.149553.

[11] P. Bhandari and H. M. Brennan, "Trustworthiness of Children Stories Generated by Large Language Models," *arXiv preprint arXiv:2308.00073*, 2023.

[12] M. Valentini, J. Weber, J. Salcido, T. Wright, E. Colunga, and K. Kann, "On the Automatic Generation and Simplification of Children's Stories," *arXiv preprint arXiv:2310.18502*, 2023.

[13] L. Ye, J. Jiang, Y. Liu, Y. Ran, D. Chang, and P. Liu, "Colin: A Multimodal Human-AI Co-Creation Storytelling System to Support children's Multi-Level Narrative Skills," in *Proc. Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, 2025, doi: 10.1145/3706599.3719837.

[14] N. Choi, P. Cyebukayire, and J. D. Choi, "Tinker Tales: Interactive Storytelling Framework for Early Childhood Narrative Development and AI Literacy," *arXiv preprint arXiv:2504.13969*, 2025.

[15] J. J. Y. Chung, M. Roemmele, and M. Kreminski, "Toyteller: AI-powered Visual Storytelling Through Toy-Playing with Character Symbols," in *Proc. CHI Conference on Human Factors in Computing Systems*, 2025, doi: 10.1145/3706598.3713435.

[16] K. T. Rosenberg, R. H. Kazi, L.-Y. Wei, H. Xia, and K. Perlin, "DrawTalking: Building Interactive Worlds by Sketching and Speaking," in *Proc. ACM Symposium on User Interface Software and Technology (UIST)*, 2024, doi: 10.1145/3654777.3676334.

[17] Google, "Read Along by Google," 2023. [Online]. Available: `https://readalong.google.com/`. [Accessed: Dec. 18, 2025].

[18] NovelAI, "AI-assisted Storytelling," 2024. [Online]. Available: `https://novelai.net/`. [Accessed: Dec. 18, 2025].

[19] Audible, "AI Narration by Audible," 2024. [Online]. Available: `https://www.audible.com/ep/audible-ai-narration`. [Accessed: Dec. 18, 2025].

[20] Talkify, "Text to Speech Online Reader," 2023. [Online]. Available: https://talkify.net/. [Accessed: Dec. 18, 2025].

[21] Story Speaker for Google Docs, "Interactive Storytelling with Google Assistant," 2023. [Online]. Available: https://workspace.google.com/marketplace/app/story_speaker/. [Accessed: Dec. 18, 2025].

[22] Y. Yu, J. Kim, Y. He, et al., "SAGA: Story Assembly Guided by AI," in *Proc. CHI Conference on Human Factors in Computing Systems*, 2024. [Online]. Available: https://saga.stanford.edu/. [Accessed: Dec. 18, 2025].

Chapter 3

Methodology and Project Plan

3.1 Methodological Approach

This project follows a design-and-build engineering methodology, which was deemed an appropriate choice for a project focused on system architecture, technical feasibility, and implementation validation. The methodology emphasizes translating a clearly defined problem into a functional prototype while maintaining a clear distinction between system components that are designed, implemented, and planned for future development.

The methodological approach is structured into four sequential stages: problem analysis and requirement definition, system design, prototype implementation, and planned validation and refinement. Each stage builds on the outcomes of the previous one and directly supports the project's primary objective of developing a dual-mode narration architecture for a child-focused storytelling application, while allowing iterative refinement across stages.

3.1.1 Stage 1: Problem Analysis and Requirement Definition

The methodology began with an analysis of existing AI-based storytelling and narration systems to identify architectural and functional limitations. This analysis revealed several gaps in current solutions, which are discussed in detail in Chapter 2 of this report. These gaps include the lack of controlled comparison between human and AI narration and limited personalization through user-initiated voice modeling.

Based on these findings, a set of functional and non-functional requirements was defined to scope the system toward a feasible capstone deliverable, as outlined in Chapter 4. At this stage, key decisions were made to prioritize system architecture concerns—namely narration handling, privacy, and offline capability—over large-scale

user evaluation or automated story generation. This step established a clear boundary between essential system capabilities and features deferred to future work.

3.1.2 Stage 2: System Design

Following requirement definition, a detailed system design was developed to specify how the system should be structured and how its components should interact. This phase focused on architectural clarity and modularity rather than feature completeness. Key design outcomes included the definition of a layered architecture separating presentation, domain logic, and data management, with strict separation between Parent View and Child View to enforce access control. Additionally, a modular narration pipeline capable of supporting both human-recorded and AI-generated audio was designed, alongside a local-first data storage strategy to support offline operation and privacy preservation.
All major system components, workflows, and data entities were designed and documented during this stage and are presented in Chapters 4 and 5. In line with the selected methodology, design decisions from earlier stages informed later refinements, reflecting standard engineering practice in early-stage system development.

3.1.3 Stage 3: Prototype Implementation

The implementation stage focused on developing a functional prototype that validates the core architectural and technical decisions proposed in the design phase. The objective of the prototype was to demonstrate feasibility rather than deliver a complete production system.
This level of implementation is appropriate for a capstone project, as it validates the primary technical contributions of the work, including the narration architecture, access control model, and privacy-aware offline design. Rather than implementing all potential features, the prototype confirms that the proposed system design can operate reliably within realistic mobile constraints.

3.1.4 Stage 4: Planned and Deferred Components

Certain components were intentionally deferred to future work in order to maintain feasibility and focus within the project timeline. These include large-scale user studies, formal experimental evaluation, advanced AI-driven comprehension assessment, cloud-based synchronization, cross-device support, and extended analytics or adaptive learning algorithms.

While some of these components are acknowledged in the system design and requirements, they were not implemented in the current prototype. Deferring these elements ensures that the project remains technically rigorous while respecting ethical, temporal, and resource constraints associated with child-focused systems.

3.2 Methodology Overview and Summary

In summary, the project methodology follows a clear and traceable progression. The process begins with problem analysis and requirement definition, followed by detailed system design, focused prototype implementation, and explicit identification of future extensions. By clearly distinguishing between designed, implemented, and planned elements, the chosen methodology ensures transparency, feasibility, and alignment with capstone expectations. This structured approach provides a coherent foundation for the design and implementation decisions presented in the subsequent chapters.

Communication and Supervision

The project was conducted under the supervision of **Dr. Fatema**, who provided academic and technical guidance throughout the development and research phases. Communication occurred primarily through weekly meetings, during which progress updates and revisions were discussed.
Feedback and meeting notes were documented and stored in a dedicated Google Drive folder titled *Supervisor Feedback* for reference and traceability.

Workflow and Version Control

To maintain an organized workflow, development and writing tasks were divided into weekly objectives aligned with the project phases. All digital assets, including source code and documentation, were backed up regularly to prevent data loss.
Each update to the prototype or report was labeled by date and version number, ensuring that prior versions remained accessible for reference and comparison.

Communication Ethics and Confidentiality

All communication and project materials were managed in compliance with academic integrity and ethical research guidelines. No personal data from participants or external

collaborators were stored within shared platforms. Access to project files was restricted to the student and supervisor to ensure confidentiality and research security.
Overall, the project management and communication strategy emphasizes transparency, consistency, and documentation discipline. Weekly meetings, clear task tracking, and structured file organization enabled smooth coordination and continuous improvement throughout both the development and research components of the project.

Chapter 4

Requirements and Assumptions

The purpose of this chapter is to establish the system requirements and constraints that will further guide the design and implementation of the storytelling application. This chapter translates the project's engineering objectives, as established in Chapter 1, into a structured and implementable set of functional and non-functional requirements. These requirements define what the system must support, how it should behave, and the conditions under which it is expected to operate, while ensuring that development proceeds in a controlled and consistent manner.

This chapter specifies how the application is organized at a system level, including its core modules, user roles, narration capabilities, and data handling approach. Clear emphasis is placed on the system's primary contribution, which is a dual-mode narration architecture that integrates human-recorded audio and AI-generated narration into a unified and modular pipeline. Supporting considerations such as secure parental controls, child-appropriate interaction flows, and privacy-preserving operation are also defined to reflect the project's child-centered and offline-capable design goals.

The requirements presented in this chapter serve as the foundation for subsequent design decisions. Each architectural element, whether related to interface separation, narration handling, or storage strategy, maps directly to the requirements outlined here, ensuring traceability between what the system is expected to do and how it is engineered.

Finally, this chapter establishes key assumptions, constraints, and design trade-offs that shape the system's scope and feasibility. By clearly defining functional expectations and operational boundaries, the chapter prepares the groundwork for Chapter 4, which details the system design and architectural structure derived from these requirements.

4.1 Functional Requirement

This section describes the core functionalities that the storytelling application must provide. Each major requirement is first defined through a brief descriptive statement, followed by a structured list of steps outlining how the system is expected to fulfill that requirement.

FR1 – User Authentication

The system shall securely authenticate guardians to access management and configuration features within the application. Authentication ensures that parental controls, narration settings, and usage limits cannot be modified by children.
Expected steps:

- The parent creates a password during the onboarding process.
- The system securely stores the password using protected local storage.
- Accessing the Parent View prompts the parent to enter their password.
- The system validates the entered password before granting access.
- If validation fails, the Parent View remains locked.

FR2 – Role-Based Access Control

The system shall enforce role-based access by separating Parent View and Child View functionalities. Ensuring that children can only access reading-related features while all configuration and management features remain entirely restricted to guardians.
Formal Access Control Constraint

```
If user_role = Child:
    deny access to {Settings, Story Approval, Narration Configuration}
Else if user_role = Parent:
    grant access to all configuration and management features
```

Expected steps:

- The system differentiates between Parent View and Child View.
- The Child View provides access to stories, favorites, and profile information only.
- The Parent View provides access to settings, narration configuration, and story creation.
- Switching from Child View to Parent View requires authentication.

FR3 – Guided On boarding and Profile Setup

The system shall provide a guided, multi-step on the on boarding flow to initialize the application for first-time use. This flow ensures that all required child and parent information is collected in a structured and user-friendly manner.
Expected steps:

- The system presents a step-by-step on boarding process with progress indicators.
- The user selects whether a setup is being performed by a parent or child. Redirecting children to their parents for set up procedures to continue and proceed further.
- The parent enters the child's name and age.
- The parent creates authentication credentials. Then are prompted to return the device to their child.
- The child selects an avatar (reading buddy).
- The child selects preferred reading genres or interests.
- The system confirms and saves all on boarding information before allowing access to the main application.

FR4 – Child Profile Management and Personalization

The system shall maintain a child profile that personalizes the reading experience. The profile stores identity information, preferences, and reading statistics.
Expected steps:

- The system associates a profile with the child's name and avatar.
- Reading preferences and interests are stored in the profile.
- The profile displays statistics such as stories read, reading streak, and favorite stories.
- Profile data is updated automatically as the child interacts with the application.

FR5 – Story Content Management

The system shall allow guardians to create and manage story content that appears in the child's library. Content visibility is controlled exclusively through the Parent View.
Expected steps:

- The parent accesses the "Create Story" option from the Parent View.
- The parent defines story metadata and content.
- The system stores the story locally.
- Approved stories become available in the child's story library.
- Unapproved or removed stories are hidden from the Child View.

FR6 – Story Library and Recommendations

The system shall display a personalized story library for the child based on profile information and selected interests. Stories are presented in a visual, card-based layout optimized for children.
Expected steps:

- The system loads available story metadata.
- Recommended stories are displayed on the home screen.
- Each story card displays the story title and page count.
- The child selects a story to begin reading or listening.

FR7 – AI Narration Generation

The system shall generate narration using AI-based text-to-speech voices. AI narration provides consistent and configurable audio playback for story content.
Expected steps:

- The system processes the story text.
- The selected AI voice profile is applied.
- Narration audio is generated or retrieved.
- The narration is prepared for playback during story reading.

FR8 – Voice Modeling and Parent Voice Recording

The system shall allow guardians to record their own voice to enable personalized narration. Recorded voice samples may be used directly or as input for AI-assisted narration.
Expected steps:

- The parent selects the “Record Your Voice” option.
- A predefined narration script is displayed.
- The system records audio samples using the device microphone.
- Recorded samples are securely stored locally.
- The recorded voice becomes available as a narration option.

FR9 – Narration Playback Control

The system shall provide narration playback controls during story reading. Playback behavior adapts to parental configuration and reading settings.
Expected steps:

- The system provides play, pause, and progress controls.
- Narration speed is adjusted based on Parent View settings.
- Optional sound effects and auto-play features are applied.
- Playback behavior follows the configured narration preferences.

FR10 – Reading Controls and Usage Limits

The system shall allow guardians to configure reading-related controls to manage the child’s usage behavior. These controls support healthy reading habits.
Expected steps:

- The parent sets a daily reading time limit.
- The system tracks reading duration.
- Reading level (Beginner, Medium, Advanced) is selected.
- The system enforces time limits and reading settings during use.

FR11 – Favorites and Reading Progress Tracking

The system shall allow children to mark stories as favorites and track reading progress across sessions.
Expected steps:

- The child marks stories as favorites.
- Favorite stories appear in the Favorites tab.
- The system tracks completed stories.
- Reading streaks and progress statistics are updated automatically.

FR12 – Local Data Storage and Persistence

The system shall store all application data locally to support offline usage and protect user privacy. Data persistence ensures continuity across sessions.
Expected steps:

- Profile data, preferences, and statistics are stored locally.
- Story content and narration audio are stored on the device.
- Stored data is retrieved upon application restart.

4.2 Non-Functional Requirement

The non-functional requirements define the quality of the attributes and constraints that govern mainly how the storytelling application operates. These requirements focus on performance, usability, scalability, and reliability. This is done to ensure the system delivers a safe, responsive, and child-appropriate experience while supporting parental control and future growth. The requirements are derived from the presented system design and the implemented prototype.

NFR1 – Performance

The system shall provide responsive interaction across all application features, including onboarding, story navigation, and audio narration.
Expected characteristics:

- Story screens shall load without noticeable delay.
- Audio narration shall start promptly after story selection.
- Playback controls (play, pause, speed adjustment) shall respond immediately to user input.
- Switching between AI narration modes shall not interrupt or restart the reading session.

NFR2 – Security

The system shall protect all user data, with particular emphasis on parental credentials and recorded voice data.
Expected characteristics:

- Parent passwords shall not be stored in plain text.
- Access to Parent View shall always require authentication.
- Recorded voice samples shall be securely stored and accessible only from Parent View.
- Children shall not be able to bypass authentication or modify parental controls.

NFR3 – Privacy and Data Protection

The system shall prioritize user privacy by minimizing data exposure and avoiding unnecessary data sharing.
Expected characteristics:

- All child profile data shall be stored locally on the device.
- No personal data shall be transmitted to third parties by default.
- Voice recordings shall remain under parental control.
- The system shall function without requiring continuous internet connectivity.

NFR4 – Usability and Child-Friendly Design

The system shall be intuitive and accessible for young users while remaining clear and efficient for guardians.
Expected characteristics:

- Interfaces shall use large icons, clear typography, and visual cues.
- Navigation shall follow a simple and consistent structure.
- The onboarding process shall guide users step-by-step with progress indicators.
- The Child View shall avoid exposing configuration or complex settings.

NFR5 – Reliability and Stability

The system shall operate reliably during extended usage and across multiple sessions.
Expected characteristics:

- The application shall not crash during story playback or narration.
- Reading progress and statistics shall persist across sessions.
- The system shall recover gracefully from unexpected interruptions.

NFR6 – Scalability

The system shall be designed to support future expansion without requiring major architectural changes.
Expected characteristics:

- Additional stories can be added without affecting performance.
- New AI voices or narration features can be integrated.
- The system can support multiple child profiles in future iterations.

NFR7 – Maintainability

The system shall be structured to allow future maintenance, updates, and feature enhancements.
Expected characteristics:

- Application components shall follow a modular design.
- Changes to narration or story features shall not impact core authentication logic.
- User interface updates shall be possible without altering data storage logic.

NFR8 – Accessibility

The system shall accommodate different reading abilities and preferences.
Expected characteristics:

- Narration speed shall be adjustable.
- Reading levels (Beginner, Medium, Advanced) shall adapt content presentation.
- Audio-based interaction shall support children with limited reading ability.

4.3 Design Alternatives and Rationale

Several alternative design approaches were considered during the system analysis and early design phases. The final design decisions were selected based on feasibility, ethical considerations, system scope, and the specific requirements of a child-focused application.

One alternative considered was a cloud-based architecture in which user profiles, stories, and narration data would be stored and synchronized using remote servers. While this approach could enable cross-device access and centralized analytics, it was not selected due to increased privacy risks, reliance on continuous internet connectivity, and higher operational complexity. A local-first architecture was chosen instead to prioritize data privacy, offline usability, and parental trust, particularly in environments involving children.

Another alternative involved the use of biometric authentication mechanisms, such as fingerprint or facial recognition, to secure access to parental controls. Although biometrics offer convenience, this approach was not adopted due to platform dependency, inconsistent hardware availability across devices, and ethical concerns related to collecting biometric data in applications involving minors. A PIN-based authentication mechanism was selected as a simpler, transparent, and privacy-conscious solution that remains accessible across devices.

For narration generation, cloud-based text-to-speech services were considered due to their advanced voice quality and scalability. However, reliance on continuous cloud processing was deemed unsuitable for offline usage scenarios and raised concerns regarding data exposure. As a result, on-device or locally managed AI narration was prioritized to maintain privacy, reduce latency, and ensure consistent availability.
These design decisions reflect deliberate trade-offs between functionality, complexity, and ethical responsibility. By explicitly evaluating and rejecting alternative approaches, the system design aligns with the project's constraints while maintaining a focus on safety, usability, and technical feasibility.

4.4 System Overview

4.4.1 Introduction

The application is implemented using React Native to support cross-platform deployment on mobile devices from a single codebase. This mobile storytelling application presents stories to children through two narration modes: pre-recorded human voice narration and AI-generated text-to-speech narration. The system is implemented as a mobile-only, deployable application and relies on an internet connection to support AI narration through a third-party service. A dual-interface design separates a secure Parent View from a simplified Child View to ensure that configuration and supervision features remain accessible only to guardians.

The Parent View supports onboarding, authentication, child profile configuration, story management, narration settings, and reading controls. Guardians can configure AI voice options, adjust narration speed, record their own voice for narration playback, and set advisory reading limits. The Child View focuses on story discovery, personalized recommendations, narration playback, favorites management, and viewing basic reading progress, with children automatically accessing this view without authentication.

The system prioritizes age-appropriate usability, privacy awareness, and maintainability. Core user data, including profiles, preferences, story content, playback progress, and voice recordings, is stored locally on the device whenever feasible, while generated narration audio may be cached after retrieval. The system does not include cloud synchronization or external analytics within its current scope.

The overall architecture follows a modular design, separating authentication, profile management, story handling, narration generation, playback control, and data storage into distinct components. This structure supports maintainability, independent testing, and future extensibility. The following sections describe the system architecture and design details that implement the functional and non-functional requirements defined in the previous chapter.

4.4.2 System Modules, Features, and User Interactions

The system is composed of several core modules that work together to deliver a controlled, age-appropriate storytelling experience. Each module has a clearly defined responsibility, allowing the system to remain modular, maintainable, and aligned with the functional requirements defined earlier.

The *Parent Module* supports onboarding, authentication, child profile configuration, story creation and approval, narration preferences, and reading controls. Through this module, guardians define how the child interacts with the application and determine which stories and narration settings are available. Access to this module is restricted through password-based authentication.

The *Child Module* provides a simplified and visually guided environment designed for young users. It allows children to personalize an avatar, browse a curated library of approved stories, mark favorites, and interact with narration playback using minimal and intuitive controls. Configuration and administrative features are intentionally excluded from this module.

The *Narration Module* manages audio preparation and playback for both pre-recorded human narration and AI-generated text-to-speech narration. It applies parent-defined settings such as voice type and narration speed and ensures consistent playback behavior during story reading.

The *Local Data Module* is responsible for storing application data on the device, including child profiles, story content, narration metadata, playback progress, and user preferences. This module supports data persistence across sessions and reduces unnecessary external data exposure.

User interactions follow a structured workflow that reflects the separation of parent and child roles. The parent installs the application and completes the initial setup by creating authentication credentials, configuring narration and reading preferences, and managing story content. Once setup is complete, the child interface becomes active. The child personalizes an avatar, explores the story library prepared by the parent, and selects a story to listen to. When a story is selected, the system retrieves the associated content, prepares narration through the Narration Module, and presents the playback interface. Throughout this process, all interactions adhere to parent-defined restrictions, ensuring consistent and predictable system behavior.

4.4.3 Overview of Workflow

The overview workflow illustrates the end-to-end operation of the system, beginning with the parent and concluding with the child listening to a story. The parent first accesses the protected Parent area to complete initial setup tasks, including child profile creation, story approval, and narration configuration. All configuration data is stored locally to ensure reliable operation without internet connectivity. Once setup is complete, the system transitions to Child mode, where the child interacts with a simplified interface to select and listen to approved stories. The narration module prepares and delivers either human-recorded or AI-generated audio based on the parent's settings, while playback progress and preferences are saved locally. Together, these steps demonstrate how the system integrates parental control, data management, narration processing, and child-friendly interaction into a cohesive workflow.

4.4.4 Ethical Considerations and Design Trade-Offs

Ethical considerations play a central role in the design of systems intended for children, particularly those that integrate artificial intelligence and audio-based interaction. While security and privacy mechanisms are implemented at a technical level, ethical responsibility also requires reflecting on the implications of design choices and their potential impact on users.

One key ethical consideration involves the use of AI-generated narration in learning environments. While AI narration improves accessibility and consistency, it may influence how children perceive authority, trust, and human presence. To mitigate this risk, the system does not replace human narration but instead supports direct comparison between human-recorded and AI-generated voices. This design choice reinforces the role of AI as a supportive tool rather than a substitute for human interaction.

Another ethical trade-off concerns data collection and analytics. Although collecting detailed behavioral data could support deeper evaluation of engagement and comprehension, such data collection was intentionally excluded from the current implementation. This decision reflects a prioritization of child privacy and consent over analytical depth, aligning with ethical standards for child-centered applications.

The decision to adopt a local-first architecture further reflects ethical judgment. By avoiding persistent cloud storage and external data transmission, the system reduces the risk of unintended data exposure and supports parental confidence in how personal and voice data are handled. While this limits certain advanced features, such as

multi-device synchronization, it strengthens trust and transparency.

Overall, the system design balances technical capability with ethical responsibility. These trade-offs demonstrate professional judgment by explicitly recognizing the social and ethical implications of AI deployment in sensitive user environments, rather than treating ethics as a purely technical concern.

Chapter 5

Design

This chapter presents the design of the proposed system. It describes the system's functional structure, user interactions, and architectural components that support the requirements defined in the previous chapter. The chapter includes use case diagrams, workflow representations, and design decisions that explain how different system modules interact to deliver a controlled and user-friendly storytelling experience.

5.1 Use Case Diagram

This use case diagram illustrates the primary interactions between the two system actors, the Parent and the Child, within the ToggleTails prototype. The Parent is responsible for managing and approving story content, configuring narration settings, recording voice samples for narration, and setting reading limits to control the child's experience. The Child interacts with the system by selecting stories, listening to narration, toggling narration modes, and viewing reading progress.

As shown in Figure 5.1, the system clearly separates parent-controlled configuration actions from child-facing storytelling interactions, reinforcing the role-based access and supervision model of the ToggleTails prototype.

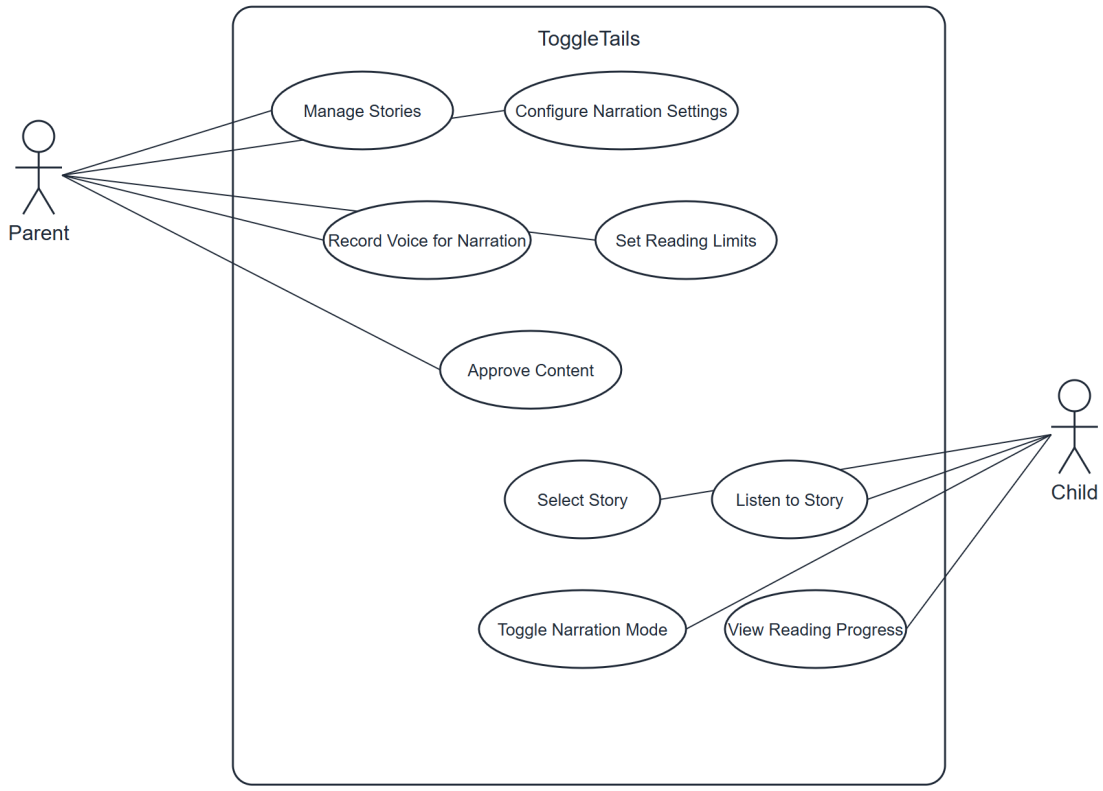

Figure 5.1: Use Case Diagram for the ToggleTails Prototype

5.2 MVC Pattern

The system design follows the Model–View–Controller (MVC) pattern to separate presentation, application logic, and data management concerns.

The **View** consists of React Native (EXPO) screens used by parents and children to display information and collect user input, without containing business logic. The **Controller** manages user interactions and application flow, enforcing access rules, handling narration selection, and coordinating system behavior based on user actions. The **Model** represents the system's data and services, including child profiles, approved stories, narration settings, local storage, audio playback, text-to-speech generation, and secure storage. Adopting the MVC pattern improves maintainability, supports clearer responsibility separation, and ensures the design remains aligned with the system requirements.

The system is composed of several major components that interact in a structured and predictable manner to support controlled storytelling and secure user interaction. At the top of the architecture, the **Presentation layer** includes all React Native (Expo) screens accessed by the parent and child, such as onboarding, permission management, the story library, and playback controls. This layer is responsible only for displaying information and capturing user input, without implementing business logic.

User actions from the interface are forwarded to the **Domain layer**, which contains the core application logic. This layer is responsible for coordinating system behavior through dedicated coordinators and use cases. For example, the Profile and Permissions Coordinator ensures that children can access only parent-approved content, the Story Library Coordinator prepares the list of available stories, and the Narration Orchestrator selects and prepares the appropriate narration mode. The Auth Manager also resides in this layer and enforces role-based access control to protect parent-specific features. The Domain layer does not store data directly; instead, it communicates with repositories to retrieve or update information as needed.

The **Data layer** manages all data persistence and retrieval operations. Repositories in this layer act as intermediaries between the Domain layer and platform-specific services, handling child profiles, story content, narration metadata, and application settings. Data persistence follows a hybrid approach: core reading state and cached content are stored locally on the device, while backend services provide centralized authentication, story generation, approvals persistence, analytics logging, and syn-

chronization when network connectivity is available. Beneath the Data layer are the **platform services**, which include local key-value storage for cached state and content, the audio engine for playback, the text-to-speech engine for AI-generated narration, and secure storage for sensitive information such as the parent PIN.
Communication between components follows a clear and consistent flow: user interactions move downward from the Presentation layer to the Domain layer, while data and service responses flow upward back to the user interface. This layered communication structure improves maintainability, supports scalability, and ensures compliance with the usability, privacy, and security requirements defined earlier in the project.

Key components of interaction include:

- The **Presentation layer** forwards user actions, such as story selection or settings updates, to the Domain layer.
- The **Domain layer** applies business rules and access constraints before requesting or modifying data.
- The **Data layer** retrieves or stores information in the local database and returns results to the Domain layer.
- The **Platform services** perform low-level operations such as audio playback, narration generation, and secure data storage.

Figure 5.2 illustrates how the Model–View–Controller pattern is applied within ToggleTails to separate user interfaces, application logic, and data management, supporting maintainability, scalability, and offline-capable operation with graceful degradation when backend services are unavailable.

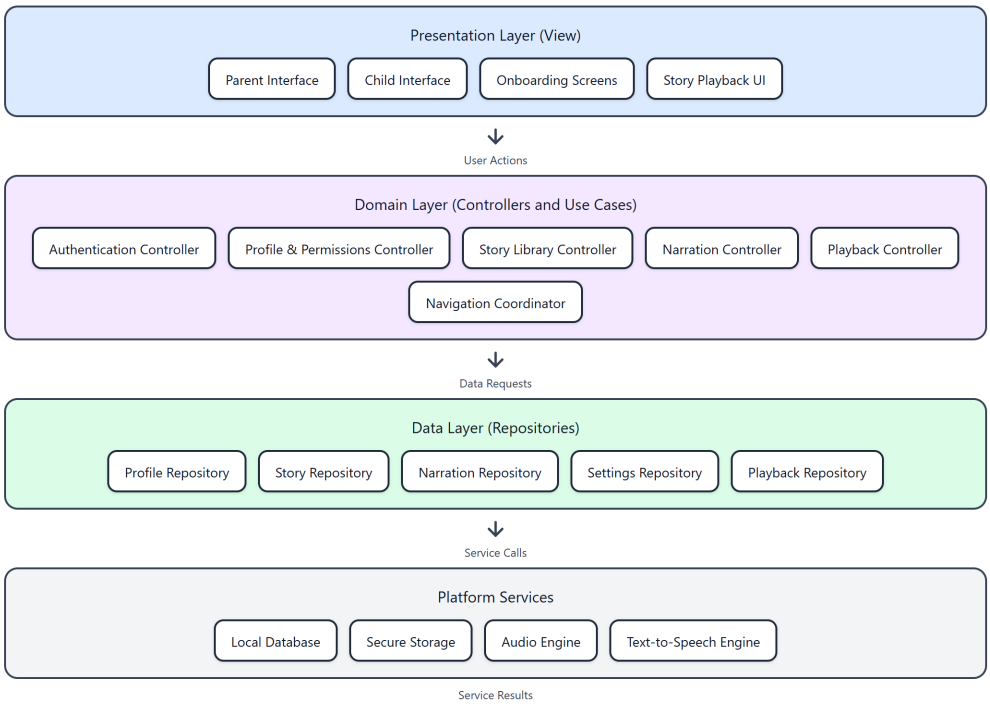

Figure 5.2: MVC-Inspired Layered Architecture of ToggleTails

5.3 Systems Components

The system is divided into several core components, each designed to satisfy one or more functional requirements. These components operate together to support dual narration, parental content control, a child-friendly user experience, and offline-capable functionality with backend-supported services for generation, approvals persistence, analytics, and synchronization.
As shown in Figure 5.3, the system is decomposed into modular components that interact in a structured manner to support authentication, content management, narration handling, playback control, and local data storage.

The **Authentication Component** protects the Parent area and ensures that only authorized users, specifically parents or guardians, can modify settings or upload stories. Its primary purpose is to enforce role-based access control and prevent children from entering restricted sections of the application. Inputs to this component include the parent's PIN or selected authentication method, while outputs include access approval signals returned to the user interface. The component interacts closely with the Profile and Permissions Coordinator and Secure Storage, where sensitive information is securely maintained. This component directly supports the functional requirements related to parental control and data security.

The **Profile and Permissions Component** is responsible for managing the child's profile and enforcing all restrictions configured by the parent. Inputs include approved stories, narration preferences, and access rules defined during the parent setup process. Outputs consist of filtered content and validated permissions that determine what the child can access within the application. This component interacts with the local database through the Profile Repository and ensures that the child's interface consistently reflects parental decisions. It fulfills functional requirements related to content filtering and personalized access.

The **Story Library Component** manages the collection of stories available to the child. Its purpose is to retrieve approved stories, maintain associated metadata, and present a curated library through the user interface. Inputs include locally stored story data and child selection actions, while outputs consist of structured story information prepared for display. This component interacts with the Story Repository and the playback interface, supporting the requirement for simple and intuitive story browsing.

The **Narration Component** handles both human-recorded and AI-generated narration. Its purpose is to prepare and deliver audio based on the selected narration mode. Inputs include story text, narration metadata, and parent-defined settings, while outputs consist of audio streams ready for playback. The component interacts with the Audio Engine when human narration is selected and with the Text-to-Speech Engine when AI narration is requested. This component directly satisfies the requirement for dual-mode storytelling.

The **Playback Component** provides the child with simple controls to listen, pause, replay, and optionally switch between narration modes. Inputs include prepared audio sources and child interaction events, while outputs include updated playback states and feedback such as progress or timing information. The component communicates with the Narration Component for audio delivery and the Settings Repository to store playback-related data, such as the last-played position. This component supports the requirement for a smooth and accessible listening experience.

Finally, the **Local Data Management Component** ensures that all profiles, stories, narration metadata, and application settings are securely stored on the device. Its purpose is to enable reliable offline operation and protect user data. Inputs include data generated during setup and interaction phases, while outputs consist of stored or retrieved information used by the Domain layer. This component interacts with all other system components through repositories and supports functional requirements related

to privacy, reliability, and offline accessibility.

System Component	Purpose	Mapped Functional Requirements	Explanation of Alignment
Authentication Component	Protect access to the Parent area and prevent unauthorized configuration changes.	FR1: Secure Parent Access FR7: Privacy & Data Protection	Ensures that only the parent can access configuration features, modify settings, upload stories, and control narration options, directly fulfilling the security and privacy requirements.
Profile & Permissions Component	Manage the child profile and enforce all restrictions defined by the parent.	FR2: Child Restrictions FR7: Privacy & Data Protection FR8: Personalized Experience	Applies parental rules, filters available content, and shapes the child interface to reflect approved preferences, supporting controlled access and personalization.
Story Library Component	Retrieve and present a curated list of parent-approved stories along with their metadata.	FR3: Story Browsing & Selection FR8: Personalized Experience	Ensures that the child can only browse and select stories approved by the parent through a simple and age-appropriate library interface.
Narration Component	Prepare and deliver story narration using either human-recorded audio or AI-generated voice.	FR4: Dual Narration Modes FR5: Child Playback & Interaction	Manages narration preparation and switching logic, ensuring the correct audio is generated or loaded and delivered consistently during playback.
Playback Component	Provide simple and accessible controls for listening, pausing, replaying, and optionally switching narration modes.	FR5: Child Playback & Interaction FR8: Personalized Experience	Supports a smooth listening experience and records playback-related preferences, such as progress and selected narration mode, to personalize future sessions.
Local Data Management Component	Store profiles, stories, narration metadata, and playback settings securely on the device.	FR6: Offline Storage & Reliability FR7: Privacy & Data Protection FR3: Story Metadata Access	Ensures all required data is stored locally, protected from unauthorized access, and available without internet connectivity, supporting reliability and safe use in educational settings.

Table 5.1: System Components Mapped to Functional Requirements

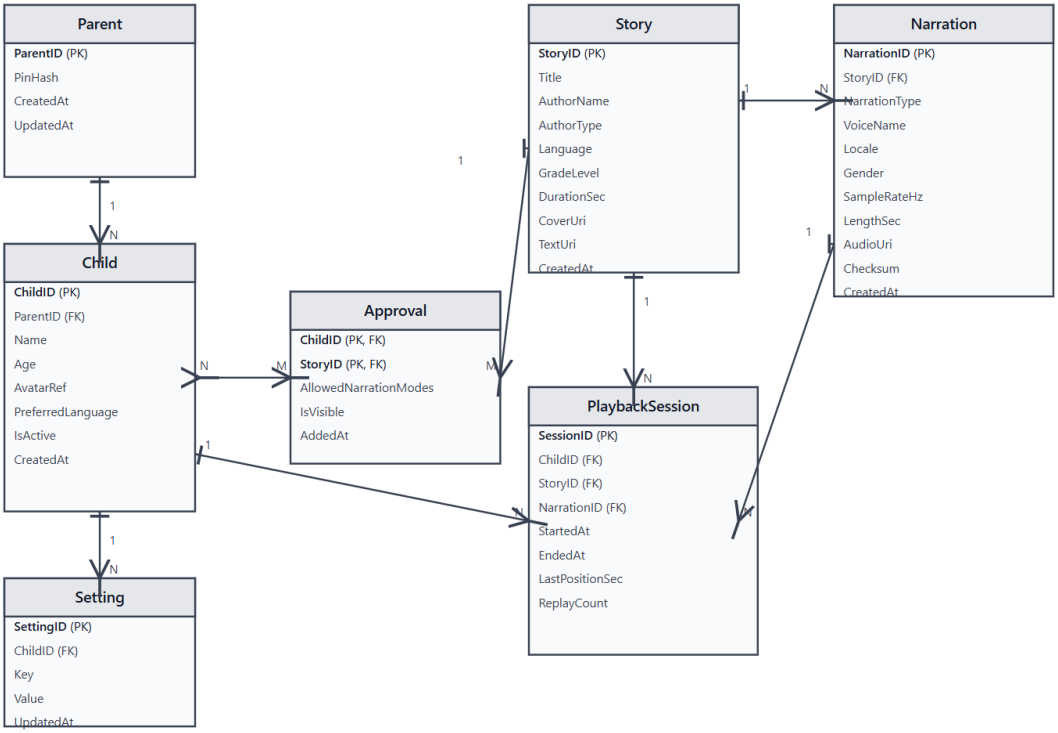

Figure 5.3: Entity Relationship Diagram for Local Storage in ToggleTails

Several additional system responsibilities are implemented as backend services and exposed via REST endpoints, including story generation, public library retrieval, approval record persistence, synchronization state reconciliation, and analytics logging.

5.4 Database Design

5.4.1 Database Model & Storage Structure

The application uses a hybrid persistence model designed to support offline-capable reading, fast access, and privacy-conscious data handling. Core reading state and cached playback can operate offline using on-device storage, while AI story generation, public library retrieval, approvals persistence, analytics logging, and synchronization rely on backend availability. Persisted server-side data is stored in MongoDB (document-based collections), while client-side state and cached content are stored locally using key-value storage.

This approach allows the system to efficiently query and retrieve data while maintaining clear relationships between entities such as parents, children, approved stories,

and available narration types.The hybrid design ensures data consistency through application-level validation, supports predictable updates via RESTful endpoints, and aligns with the modular architecture by separating responsibilities between local caching and server persistence. By combining local storage with optional cloud persistence, the system supports both parent configuration workflows and the child's story consumption experience while enabling offline-capable operation.

5.4.2 Main Entities, Attributes, & Relationships

Below are the core entities that support parent setup, curated access, dual narration, playback, and offline storage. Relationships between entities are represented using identifier references (e.g., parentId, childId, storyId) and enforced through application-level ownership checks and validation logic.

1. **Parent**
   - **Attributes:** ParentID (**PK**), PinHash, CreatedAt, UpdatedAt
   - **Purpose:** Represents the parent or guardian who owns and protects configuration settings for one or more children.
2. **Child**
   - **Attributes:** ChildID (**PK**), ParentID (reference to Parent), Name, Age, AvatarRef, PreferredLanguage, IsActive, CreatedAt
   - **Purpose:** Represents the child user and stores profile details and preferences.
3. **Story**
   - **Attributes:** StoryID (**PK**), Title, AuthorName, AuthorType (Human), Language, GradeLevel, DurationSec, CoverUri, TextUri, CreatedAt
   - **Purpose:** Stores the master record for each story available in the application.
4. **Narration**
   - **Attributes:** NarrationID (**PK**), StoryID (reference to Story), NarrationType (Human or AI), VoiceName, Locale, Gender, SampleRateHz, LengthSec, AudioUri, Checksum, CreatedAt
   - **Purpose:** Stores one or more narration versions per story, including human recordings and AI-generated narration.

5. **Approval (Association Record)**
   - **Attributes:** ChildID (reference to Child), StoryID (reference to Story), AllowedNarrationModes, IsVisible, AddedAt
   - **Uniqueness:** Enforced using a compound uniqueness constraint on (ChildID, StoryID) at the persistence layer to prevent duplicate approval records.
   - **Purpose:** Controls which stories a child can access and which narration modes are permitted.
6. **PlaybackSession**
   - **Attributes:** SessionID (**PK**), ChildID (reference to Child), StoryID (reference to Story), NarrationID (reference to Narration), StartedAt, EndedAt, LastPositionSec, ReplayCount, EngagementFlag (optional), Rating (optional)
   - **Purpose:** Stores minimal on-device playback data to support resuming stories and basic interaction tracking.
7. **Setting**
   - **Attributes:** SettingID (**PK**), ChildID (reference to Child), Key, Value, UpdatedAt
   - **Purpose:** Stores small key-value preferences such as default narration mode, accessibility options, or interface settings.
8. **PermissionRule**
   - **Attributes:** RuleID (**PK**), ChildID (reference to Child), CanSwitchNarration, MaxDailyMinutes, QuietHoursStart, QuietHoursEnd, UpdatedAt
   - **Purpose:** Encodes parent-defined rules that shape and restrict the child's experience.

Core Relationships (Cardinalities)

- **Parent 1 to N Child**: One parent can manage multiple children, while each child belongs to exactly one parent.
- **Child N to M Story (via Approval)**: A child may be approved for multiple stories, and a story may be approved for multiple children. The Approval record stores allowed narration modes for each pairing.
- **Story 1 to N Narration**: A story can have multiple narration tracks, including human recordings and AI-generated voices.

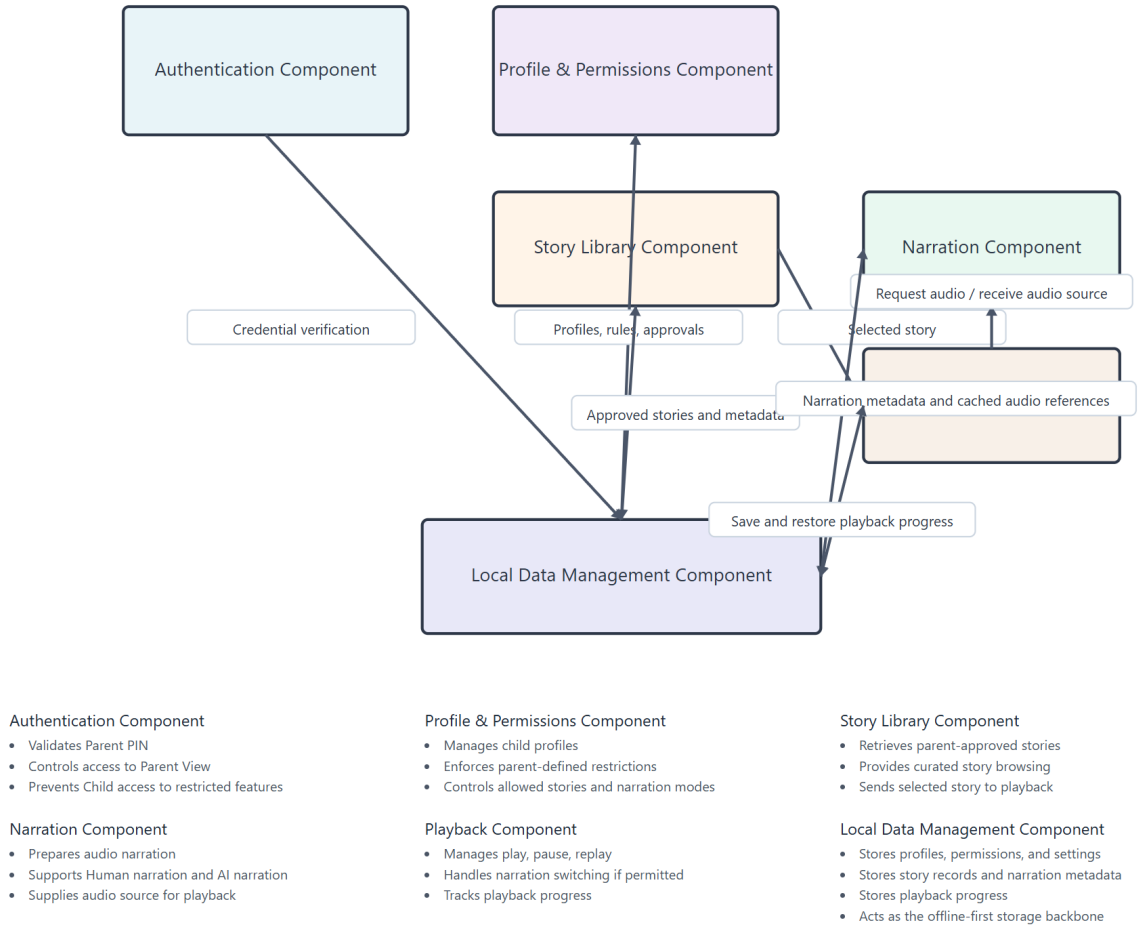

Figure 5.4: Major System Components and Their Interactions in ToggleTail

- **Child 1 to N PlaybackSession and Narration 1 to N PlaybackSession**: Each playback session links a child, a story, and the selected narration version.
- **Child 1 to N Setting and Child 1 to N PermissionRule**: A child may have multiple preferences and permission rules applied.

Figure 5.4 presents the entity relationship diagram for the local storage layer, illustrating how parent profiles, child profiles, stories, narration metadata, playback sessions, and settings are structured and related to support offline operation and data integrity.

5.4.3 Constraints, Normalization, and Indexing Strategy

The persistence layer applies constraints to ensure data accuracy, consistency, and secure handling of user information. Records are uniquely identified using document identifiers, while referential integrity between related entities (e.g., parent, child, story,

approval) is enforced through application-level ownership checks, validation logic, and compound uniqueness constraints where appropriate (e.g., per-child approvals). Additional domain constraints enforce valid values for fields such as narration type and playback position. Sensitive data, including the parent authentication PIN, is stored in a protected format to support privacy and security within the local device environment.

The data model follows a structured, modular design appropriate for a document-based database. Data is separated into logical collections and local storage keys to reduce redundancy while keeping retrieval efficient for common application workflows. This design reduces redundancy and prevents update anomalies. Stories, narrations, approvals, and playback sessions are stored independently to avoid duplication and maintain clean relationships. Limited and controlled denormalization may be applied for performance reasons, such as caching derived metadata, but the overall model prioritizes clarity, maintainability, and correctness.

To ensure strong performance on mobile devices, the indexing strategy focuses on common access patterns. Indexes are defined at the collection level (e.g., parentId, childId, storyId, and compound indexes on childId + storyId) to accelerate common access patterns such as parent-to-child retrieval, story filtering by metadata (e.g., reading level), and approval lookups. Additional targeted indexes, such as indexing stories by title or playback sessions by child and story, support fast access to the curated library and efficient playback resumption. By limiting indexes to essential paths, the system balances read performance with storage efficiency and lightweight write operations, ensuring a responsive experience during both parent configuration and child playback.

Chapter 6

System Implementation

6.1 Introduction to the Implementation

This chapter presents the implementation of ToggleTails as a hybrid client–server mobile application designed to support controlled storytelling, AI-assisted narration, and parent-governed content management. The implementation operationalizes the architectural design defined in Chapter 5 and translates it into a functioning system composed of a React Native mobile client, a Node.js/Express backend API, and optional MongoDB Atlas cloud persistence. The objective of this chapter is to provide a detailed technical account of how the system was constructed and how the implemented components align with the functional, security, and ethical requirements established earlier in the project.

The current system supports end-to-end interaction flows including structured onboarding, child profile creation and management, curated story browsing, AI-based story generation, dual narration modes, and parent-restricted configuration access enforced through authentication and PIN validation. The backend exposes RESTful endpoints responsible for authentication, story generation, library retrieval, analytics logging, and child profile management. Although the system is presently executed within a local development environment, the architecture is structured to support cloud deployment without structural modification, reflecting forward compatibility with production infrastructures.

The mobile application is built using React Native within the Expo framework, enabling cross-platform development while retaining access to native device capabilities such as audio playback, speech recognition, and secure storage. The backend runs locally using Node.js and Express and is configured to connect to MongoDB Atlas for persistent storage when a valid connection string is provided. In the absence of a configured database, the system degrades gracefully by operating in a local caching mode, ensur-

ing continued functionality during development and partial offline operation.
Core modules implemented within this system include authentication and role-based access control, onboarding and child profile configuration, story library retrieval and filtering, AI-powered story generation using OpenAI models, dual narration modes through text-to-speech and recorded parent narration, speech-recognition-assisted reading with fuzzy word matching, offline caching for approved stories and playback state, and basic analytics event logging.
While the application is not yet deployed to a public production environment, the implementation reflects a complete structural realization of the system architecture, with environment configuration, database integration capability, and native build support prepared for deployment through infrastructure provisioning rather than architectural redesigning.
The following sections provide a detailed technical breakdown of the system structure, technology stack, data management strategies, API design, testing approach, security considerations, and implementation decisions that collectively demonstrate the feasibility and robustness of the developed solution.

6.2 Technology Stack

The implementation of ToggleTails utilizes a modern full-stack JavaScript ecosystem structured around a hybrids mobile and REST-based backend architecture. The selected technology stack was chosen to balance cross-platform compatibility, rapid development, modular scalability, and support for AI-assisted functionality.

6.2.1 Mobile Application Layer

The mobile client is implemented using React Native version 0.81.5 within the Expo SDK 54.0.33 frameworks. React Native was selected to enable a single cross-platform codebase targeting both Android and iOS environments while preserving access to native device capabilities such as audio playback and speech recognition. TypeScript was incorporated to reduce runtime errors, improve development tooling, and ensure stronger compile-time validation of application logic.
Text-to-speech functionality is implemented using expo-speech (version 14.0.8), allowing narration to be performed directly on the device without requiring server-side audio generation. Speech recognition for the “Help Me Read” mode is implemented using expo-speech-recognitions (version 3.1.0). Because speech recognition relies on native device permissions and platform-specific dependencies, native builds are required through Expo Application Services (EAS). Local caching and state persistence

are handled using @react-native-async-storage/async-storage (version 2.2.0). Audio recording for parent narration is supported through expo-file-system (version 19.0.21), enabling recorded narration files to be stored and replayed directly from the device.

6.2.2 Artificial Intelligence Integration

AI-powered story generation is implemented using the OpenAI SDK (version 4.28.0) with the gpt-4o-mini model. Direct API calls are made from the backend to generate age-appropriate, interest-based story content dynamically. The selected model balances output quality with computational efficiency and cost considerations. Generated stories are structured and validated before being returned to the mobile application and may be persisted in MongoDB when database connectivity is enabled.

6.2.3 Testing Frameworks

Backend testing is conducted using Jest (version 30.2.0) in conjunction with Supertest (version 7.2.2)) to validate API endpoints, middleware behavior, and database interactions. These tests focus primarily on authentication flows, route protection, story retrieval logic, and integration with MongoDB. Frontend automated testing is currently limited due to dependency conflicts between React 19 and React Native Testing Library. However, type safety is enforced through TypeScript compilation using tsc with no-emit mode, and structured manual validation was conducted to verify interaction flows, narration modes, and caching behavior. The jest-expo preset is configured but requires additional stabilization.

6.2.4 Development and Build Tooling

Version control is managed using Git, with structured commits maintained throughout development. Native builds are configured using Expo Application Services (EAS), with development, preview, and production profiles defined in the eas.json configuration file. ESLint (version 9.25.0) with eslint-config-expo (version 10.0.0) is used to enforce coding standards and detect potential errors during development. TypeScript provides additional compile-time validation to improve code reliability. While a dedicated formatting tool such as Prettier is not currently configured, formatting consistency is maintained through ESLint integration.
Overall, the selected technology stack reflects deliberate engineering decisions focused on modularity, security, AI integration capability, cross-platform deployment, and

long-term scalability while remaining realistic within the constraints of a capstone implementation.

Layer	Technology
Mobile	React Native (Expo SDK 54)
Backend	Node.js + Express
Database	MongoDB Atlas
AI	OpenAI GPT-4o-mini
Auth	JWT + bcrypt
Testing	Jest + Supertest

Table 6.1: Technology Stack Overview

6.3 System Architecture and Structure

ToggleTails is implemented using a hybrid client–server architecture combined with a layered internal structure within the mobile application. This approach extends the MVC-inspired design presented in Chapter 5 into a fully operational system while maintaining separation of concerns between presentation, business logic, data management, and external services.

6.3.1 Architectural Pattern

The overall architecture follows a hybrid client–server model with layered separation inside the mobile client. The backend is required for specific capabilities such as AI-based story generation and dynamic story library retrieval. However, core functionality such as onboarding, child profile management, cached story reading, and parent-recorded narration can operate independently of continuous backend connectivity. This mixed dependency model allows the system to balance cloud-powered intelligence with partial offline reliability. This implementation realizes the layered client–server architecture described in Chapter 5.

Within the mobile application, a layered structure is applied to maintain separation of concerns and improve maintainability.

The **Presentation layer** is implemented using React Native with Expo Router's file-based navigation system and includes screens, reusable UI components, and navigation logic such as stack and tab navigators. This layer is responsible for rendering both

parent and child interfaces and managing user interaction events.

The **Application Logic layer** contains the core business logic of the system. This includes narration mode selection, story filtering based on approvals, role-based navigation enforcement, playback control, and reading progress management.

The **Data layer** is responsible for data retrieval and persistence, abstracting backend communication through the api/ module, handling local storage through dedicated wrappers in the storage/ module, and centralizing configuration such as API endpoints within the config/ module. This layer manages interaction with both AsyncStorage and the backend API.

The **Platform Servicess** layer provides controlled access to device-level capabilities, including text-to-speech via expo-speech, speech recognition via expo-speech-recognition, file storage through expo-file-system, and network communication. User actions originate in the Presentation layer, are processed by the Domain layer, and then interact with either local storage or backend services through the Data layer, with responses propagating upward for rendering in the user interface.

At a high level, essentially the system consists of two primary subsystems: a React Native mobile application and a Node.js/Express backend API. The mobile application is responsible for user interaction, device-level functionality, and local caching, while the backend provides centralized services including AI-powered story generation, story library retrieval, authentication, and optional cloud persistence through MongoDB Atlas.

6.3.2 Mobile Application Structure

The mobile codebase is organized into modular directories to ensure clarity and maintainability. Screens are defined in the app/ directory using Expo Router's file-based routing system, while reusable interface elements are placed in the components/ folder. Backend communication is centralized within the api/ module, and local persistence is abstracted through the storage/ layer, which wraps AsyncStorage operations. Supporting utilities are located in services/, configuration values such as the API base URL are managed in config/, and reusable logic is encapsulated within custom hooks in the hooks/ directory.

Navigation is implemented using Expo Router with both stack and tab patterns. The application avoids centralized state management libraries, instead relying on component-level state combined with AsyncStorage for persistence, reducing architecturals complexity while remaining sufficient for the current scope.

6.3.3 Backend Architecture

The backend subsystem is implemented using Node.js and Express and follows a modular, feature-based routing structure. Each major functional domain is encapsulated within its own router module, including authentication (routes/auth.js), child management (routes/children.js), story operations (routes/stories.js), analytics logging (routes/-analytics.js), text-to-speech handling (routes/tts.js)„ health checks (routes/health.js), approvals (routes/approvals.js), and synchronization (routes/sync.js). This feature-based separation improves maintainability, enables clearer responsibility boundaries, and supports future extensibility without introducing cross-module coupling.

Middleware is applied centrally to enforce consistent request handling behavior. Cross-Origin Resource Sharing (CORS) is enabled globally using app.use(cors()), allowing flexible development access during local execution.

Authentication is implemented using JSON Web Tokens (JWT). During registration, parent passwords are securely hashed using bcryptjs before storage. The backend issues signed JWTs upon successful authentication, enabling stateless authorization for protected routes. Currently, JWTs are validated for endpoints such as /api/children, /api/approvals, and /api/sync. However, token persistences on the mobile client is not yet fully integrated, and role claims are not explicitly encoded beyond a parent identifier. Parent PINs used for client-side access control are stored locally within AsyncStorage and are not currently hashed, reflecting a development-stage implementation decision.

Data persistence is handled using MongoDB Atlas when a valid Mongo DB is provided. Mongoose is used as the object data modeling layer, with schemas defined for Parent, Child, Story, Approval, Analytics, Narration, and PlaybackSession collections..The schemas are defined in plain JavaScript without strict TypeScript typing, providing structural validation while maintaining implementation simplicity. If the database connection string is not configured, the server logs a warning and continues execution, allowing partial functionality without persistence. Routes requiring database access return controlled errors when no connection is established.

Story library data can be populated using a dedicated seeding mechanism. A script (scripts/seedLibrary.js) and an associated endpoint (POST /api/stories/seed-library) allow structured story records to be inserted into the database. AI-generated stories are produced through the /api/stories/generate endpoint, which integrates with the OpenAI SDK. This endpoint returns structured JSON responses containing a success flag and a story object with fields including title, full text, paginated content, reading level classification, and thematic tags. Returning structured data rather than raw text enables consistent client-side rendering, filtering, and metadata processing.

Overall, the backend architecture reflects a modular and extensible design that supports authentication, structured AI integration, analytics logging, and optional cloud

persistence while remaining compatible with local development execution..

6.4 Core Components and Features

6.4.1 Authentication Component

The Authentication component provides secure parent access to restricted features such as story approvals, child profile management, and synchronization. It's primary purpose is to enforce role separation between parent and child usage by requiring parent authentication while allowing children to access the reading experience without account credentials. Inputs to this component include parent registration and login data (email and password), PIN credentials for quick access, and bearer tokens supplied through these HTTP Authorization header for authenticated requests. The component outputs structured authentication responses containing a success flag, a signed JWT token, and a sanitized parent object; for protected requests, the authentication middleware enriches the request context by attaching req.parentId and req.parent.
Internally, registration validates required fields, enforces a minimum password length, checks for duplicate accounts using a MongoDB lookup, and hashes credentials using bcrypt (genSalt and hash) before persistence. Login follows a similar pattern by locating the parent account using a normalized email and verifying the password via bcrypt comparison. JWT issuance is performed using a signing function that embeds parentId with an expiry window (7 days), enabling stateless authorization. For route protection, middleware extracts and verifies the bearer token, resolves the parent identity from the database, and rejects invalid or expired tokens with appropriate HTTP status codes. The component interacts primarily with the Parent Mongoose model for persistence and is applied as middleware for endpoints such as /api/children, /api/approvals, and /api/sync, while optional authentication is used for generation endpoints where login is not strictly required. Current limitations include reliance on a development fallback JWT secret if environment configuration is missing, lack of brute-force protection (no rate limiting), absence of email verification and password recovery flows, and client-side token storage not yet fully integrated. Additionally, parent PIN handling remains development-stage and requires strengthening for production security.

6.4.2 Story Generation Component

The Story Generation component produces personalized, age-appropriate stories on demand using OpenAI's API. Its objective is to adapt narrative complexity to the child's reading level and interests to support engagement and, literacy development. Inputs

include child profile attributes (name, age, reading level), a list of selected interests, optional custom prompts and titles, and, an optional persistence flag indicating whethers the story should be stored. The output is a structured JSON payload containing a success flag and a story object with a title, full story text, paginated pages, a reading level label, and derived theme metadata; where persistence is enabled and authentication is present, the component may return the stored MongoDB documents.
All while the endpoint applies defaults for missing inputs, verifies that the OpenAI API key is available, and maps the selected reading level to predefined constraints. Constraints can include target vocabulary complexity, word count range, and page count range. Interests are translated into story themes using a deterministic mapping to ensure consistent prompt conditioning. A system prompt enforces writing constraints, while a user prompt injects personalization variables such as child name and themes. The backend calls the OpenAI chat completion endpoint using the gpt-4o-mini model and post-processes the returned text by splitting it into pages based on paragraph separation. Current constraints include lack of request validation for free-form prompt fields, no explicit prompt-injection mitigation, absence of rate limiting (making abuse possible), and cost overhead due to multiple API calls per generated story. The generation process also requires network connectivity, with no offline AI generation capability.

6.4.3 Library and Approval Filtering Component

The Library and Approval Filtering component manages access to curated story content from an online resource by OpenAI known as storyweaver which further enforces the stories displayed to the child are with respect and reference to child safe content. Currently, its purpose is to ensure children only view content that is pre-vetted or explicitly approved by a guardian, satisfying supervision and safeguarding requirements. However once StoryWeaver is integrated in full that might be adjusted. Currently, Inputs include library retrieval parameters (age band, reading level, category, pagination controls), and approval operations containing identifiers (childId, storyId), approval state, and permitted narration modes. And consequently, the outputs include paginated story library responses containing story metadata and, for approval operations, an approval object reflecting visibility, allowed modes, and parent attribution.

Internally, library retrieval constructs a MongoDB query constrained to library-sourced stories and conditionally applies filters based on provided parameters.Approvals are stored using an upsert strategy (findOneAndUpdate with upsert: true) to toggle state without creating duplicates, supported by a compound uniqueness constraint over (childId, storyId). Noticably, the key constraints include lack of text search capabilities, ab-

sence of backend caching (each library request hits MongoDB directly), and per-child approval overhead (the same story must be approved separately for each child). Bulk approvals are limited, and library seeding relies on manual or scripted population rather than automated ingestion pipelines.

6.4.4 Narration System (Native TTS and Parent Recording

The Narration System enables audio-based storytelling through on-device text-to-speech and parent-recorded narration, supporting accessibility and engagement for early readers. Its purpose is to provide multiple narration modalities and allow guardians to personalize stories using their own recorded voice. Inputs include story text and playback configuration for native TTS such as the rate, pitch and language. With respect to restricted control of the voice selection. and recording-specific metadata for parent narration namely storyId, page index, file URIs, duration, and completion state. Consequently, the outputs include real-time audio playback for TTS, recorded audio files stored on the device, and persistent recording indexes that enable retrieving and replaying narration per story and per page.

6.4.5 Speech Recognition and Fuzzy Matching (Help Me Read)

The Speech Recognition and Fuzzy Matching component implements interactive “Help Me Read” functionality by listening to a child’s spoken reading and validating pronunciation against on-screen words. Inputs would mainly include an audio stream captured from the microphone through expo-speech-recognition. With intention to then have the active story page text tokenized into expected word sequences, and the current word index tracking reading position. Outputs include real-time UI highlighting for the active target word, correctness feedback messages, a running completion counter, and corrective audio playback of the expected word when mismatch occurs.

Internally, the component updates the expected word list whenever the page changes by splitting page text into tokens and resetting progress state. When listening begins, permissions are requested and continuous speech recognition is started with interim results enabled. Recognition events stream transcripts to a processing function that normalizes text (lowercasing and punctuation removal) and compares spoken tokens to the expected token at the current index. Correct matches advance the index and update progress; mismatches trigger corrective feedback and optionally speak the correct word.

6.5 API Documentation

The ToggleTails backend exposes a modular RESTful API designed to support authentication, child management, story generation, narration services, synchronization, approvals, and analytics logging. The API follows a structured request–response format using JSON payloads and enforces role-based access control through JWT authentication middleware. Each functional domain is encapsulated within its own router module to maintain separation of concerns and extensibility.

6.5.1 Endpoint Organization

The backend is organized into the following primary modules:

- Authentication Module
- Child Profile Management Module
- Story Management and AI Generation Module
- Approvals and Moderation Module
- Synchronization Module
- Text-to-Speech (TTS) Module
- Analytics Module

Each module exposes a defined set of HTTP endpoints as described below.

6.5.2 Authentication Module

The authentication module manages parent account creation, login, token issuance, and credential updates.
Endpoints:

- POST /api/auth/register
- POST /api/auth/login
- POST /api/auth/pin
- GET /api/auth/me
- PATCH /api/auth/pin

- PATCH /api/auth/password

JWT tokens are issued upon successful authentication and must be included in the Authorization header for protected routes using the Bearer scheme. The requireAuth middleware validates tokens and attaches the resolved parent identity to the request context.

6.5.3 Child Profile Management Module

This module enables authenticated parents to create, retrieve, update, and delete child profiles.
Endpoints:

- POST /api/children
- GET /api/children
- GET /api/children/:id
- PATCH /api/children/:id
- DELETE /api/children/:id

All endpoints require authentication and validate ownership by matching the child record to the authenticated parent identifier.

6.5.4 Story Management and AI Generation Module

The stories module supports manual story management and AI-powered story generation.

Manual Story Management

- GET /api/stories/library
- GET /api/stories/:id
- POST /api/stories
- DELETE /api/stories/:id

AI Story Generation

Endpoint:
POST /api/stories/generate
The endpoint integrates with the OpenAI SDK using the gpt-4o-mini model. Prompt construction incorporates:

- Child age
- Reading level (Beginner, Intermediate, Advanced)
- Selected interests
- Optional custom title

The response is structured as:

```
{
  "success": true,
  "title": "...",
  "text": "...",
  "story": {
    "id": "...",
    "title": "...",
    "text": "..."
  }
}
```

If AI generation fails, the endpoint returns a fallback indicator enabling the client to degrade gracefully.

6.5.5 Approvals and Moderation Module

This module enforces parental control over story visibility.
Endpoints:

- POST /api/approvals
- GET /api/approvals
- GET /api/approvals/child/:childId
- PATCH /api/approvals/:id/favorite

- DELETE /api/approvals/:id
- POST /api/approvals/bulk

Approval records are stored per child and determine story visibility and permitted narration modes.

6.5.6 Synchronization Module

The synchronization module supports offline-first state reconciliation between the mobile client and backend.
Endpoints:

- GET /api/sync/pull
- POST /api/sync/push
- GET /api/sync/status

Synchronization is implemented using timestamp-based delta updates. Push requests include structured action payloads such as:

```
{
  "actions": [
    {
      "type": "UPDATE_CHILD",
      "timestamp": "...",
      "payload": { ... }
    }
  ]
}
```

6.5.7 Text-to-Speech Module

The TTS module integrates with the ElevenLabs API for server-side audio generation.
Endpoints:

- GET /api/tts/voices
- POST /api/tts/generate
- POST /api/tts/stream

If the ElevenLabs API key is not configured, the backend returns HTTP 503, enabling the client to fall back to native on-device TTS. This is done with the intention to be weary of the future advancements and potential of different entities not being able to access internet.

6.5.8 Analytics Module

The analytics module supports lightweight event logging and feature flag management.
Endpoints:

- POST /api/analytics/event
- GET /api/analytics/stats
- GET /api/analytics/features
- POST /api/analytics/features

No personally identifiable child data is transmitted by default.

6.5.9 Request and Response Structure

All API responses follow a standardized JSON structure:

```
{
  "success": true | false,
  "data": { ... },
  "error": {
    "message": "...",
    "statusCode": ...
  }
}
```

Client errors return HTTP 400-series codes, while unexpected server failures return HTTP 500.

6.5.10 Middleware and Error Handling

Authentication enforcement is implemented through the requireAuth middleware, which:

- Extracts the Bearer token
- Verifies JWT signature and expiration
- Resolves the parent record
- Rejects invalid requests with HTTP 401

Centralized error handling is implemented via errorHandler.js, which standardizes error responses and suppresses stack traces in production mode. A notFoundHandler middleware ensures undefined routes return structured 404 responses.

6.6 Error Handling and Logging

ToggleTails implements centralized error handling to ensure consistent responses and controlled failure behavior across the backend. A global error-handling middleware (errorHandler.js) captures application errors and standardizes responses. The middleware determines the appropriate HTTP status code (defaulting to 500 if unspecified), logs the error, and returns a structured JSON response containing a success flag and an error object. In production mode, internal details are suppressed to prevent information leakage.
A dedicated notFoundHandler middleware captures unmatched routes and forwards them as operational errors using a custom ApiError utility. This ensures that undefined endpoints return a consistent 404 JSON response rather than crashing the server.
Within route handlers (e.g., stories.js), business logic is wrapped in try/catch blocks. Validation errors return appropriate client-side status codes such as 400, while unexpected failures return 500 responses. Errors are consistently returned in JSON format: "success": false, "error": "message": "Resource not found", "statusCode": 404
Server-side errors are logged using a custom logging utility (logger.js), which wraps console.error and includes contextual metadata such as HTTP method, URL, and status code. This approach improves traceability during debugging while maintaining structured client-facing error responses.

6.7 Security Considerations

6.7.1 JWT-Based Authentication Mechanism

ToggleTails implements stateless authentication using JSON Web Tokens (JWT) to secure parent-restricted backend endpoints. Upon successful login or registration, the

```
ParentSchema.methods.setPassword = async function(password) {
  const salt = await bcrypt.genSalt(10);
  this.passwordHash = await bcrypt.hash(password, salt);
};

ParentSchema.methods.verifyPassword = async function(password) {
  return bcrypt.compare(password, this.passwordHash);
};
```

Figure 6.1: Code Sample

backend generates a signed token containing the authenticated parent's unique identifier (parentId). The token is signed using a secret key and configured with currently a seven-day expiration period.
Token generation is implemented in `backend/src/middleware/auth.js` as shown in Listing 6.1.

Listing 6.1: JWT Token Generation Function

```
const generateToken = (parentId) => {
  return jwt.sign(
    { parentId },
    JWT_SECRET,
    { expiresIn: JWT_EXPIRES_IN }
  );
};
```

Each authenticated request must include the token in the HTTP Authorization header using the Bearer scheme. The requireAuth middleware verifies the token, ensures its validity and expiry status, retrieves the corresponding parent record from MongoDB, and attaches the resolved parent object to the request context (req.parent and req.parentId). If verification fails, the middleware returns an HTTP 401 Unauthorized response.
This approach enables stateless authorization, supports horizontal scalability, and avoids server-side session storage.

6.7.2 Password and PIN Hashing

Sensitive credentials are never stored in plaintext. Parent passwords are hashed using the bcryptjs library before being persisted to the MongoDB database. Parent PIN handling is currently split between backend hash storage (pinHash) and client-side local validation, reflecting a development-stage implementation that would require further hardening for production deployment.
The Parent model defines instance methods for hashing and verification:

The use of bcrypt with salting mitigates rainbow table attacks and significantly increases resistance to brute-force password cracking. PIN values are similarly stored as hashes (pinHash), ensuring that even in the event of database exposure, plaintext credentials are not recoverable.

6.7.3 Enviroment Variable Management

Sensitive configuration values such as the JWT secret are externalized through environment variables. The backend loads environment variables at startup using the dotenv package:
require('dotenv').config(path: path.join($_{dirname,'..','.env'}$));
This prevents secret values from being hardcoded in source files and supports secure deployment practices. A development fallback secret is defined to ensure local execution, but this is explicitly intended for non-production environments.

6.7.4 Route-Level Access Control

Authentication enforcement is implemented at the route level using middleware composition. The following endpoint groups are protected using the requireAuth middleware::
Authentication Management GET /api/auth/me PATCH /api/auth/pin PATCH /api/auth/-password
Child Management POST /api/children GET /api/children PATCH /api/children/:id DELETE /api/children/:id
Story Management POST /api/stories DELETE /api/stories/:id GET /api/stories/parent/all
Synchronization Services GET /api/sync/pull POST /api/sync/push GET /api/sync/status
This ensures that only only authenticated parents can access or modify sensitive resources associated with their accounts.

6.8 Parent PIN Storage Strategy

The parent PIN serves as a local “Parent Gate” mechanism within the mobile application to restrict access to configuration and approval settings.
On the backend, the PIN is stored securely as a hash in the Parent collection. On the frontend, the PIN is stored locally using React Native's AsyncStorage, under a defined key: export const PIN$_KEY =' parent_pin_v1';$

Storing the PIN locally enables fast offline validation without requiring repeated backend calls. However, AsyncStorage does not provide hardware-level encryption, which introduces a moderate security trade-off for convenience. For production deployment, the use of encrypted storage mechanisms (e.g., Expo SecureStore) would be recommended.

6.8.1 Identified Security Limitations

Although the implemented security mechanisms are appropriate for a capstone-level prototype, several limitations remain and are explicitly acknowledged. The backend currently does not implement rate-limiting middleware (such as express-rate-limit), which means authentication endpoints including /api/auth/login and /api/auth/pin are not protected against brute-force or credential-stuffing attacks.
Additionally, Cross-Origin Resource Sharing (CORS) is enabled globally using app.use(cors()) without restricting allowed origins. While this configuration simplifies local development and testing, further down the line it would require domain-specific restrictions in a production environment to prevent unauthorized cross-origin access.
The system also operates over HTTP during development, meaning transport-layer encryption is not enforced. In a production deployment, HTTPS would need to be configured through a reverse proxy (e.g., Nginx) or a managed cloud service to ensure encrypted communication between clients and the server. Furthermore, security-related HTTP headers (such as those provided by the helmet middleware) are not currently implemented, leaving the application without additional protections against common web vulnerabilities such as clickjacking or MIME-type sniffing. Finally, JWT authentication tokens are stored on the client device using React Native's AsyncStorage. Although this approach is common in mobile development, it does not provide hardware-backed encryption and therefore offers a lower security guarantee compared to secure storage mechanisms such as encrypted keychain or secure enclave solutions.
These limitations are documented to demonstrate awareness of production-grade security requirements and to identify clear areas for future hardening and system enhancement prior to public deployment.

6.8.2 Summary of Security Implications

The security model of ToggleTails combines JWT-based stateless authentication, bcrypt credential hashing, route-level authorization enforcement, and environment-variable-based secret management. While ultimately setting up suitable for a prototype and development deployment, additional production hardening measures. Which

include rate limiting, secure headers, restricted CORS, HTTPS enforcement, and encrypted token storage as further development that would be required prior to public release.

6.9 Dependencies and Package Management

ToggleTails uses npm as its package manager, with a package-lock.json committed in basically both the frontend and backend directories to ensure reproducible builds and dependency version consistency.

6.9.1 Backend Dependencies

The backend is built using Node.js and Express (v4.18.2). Core production dependencies include:
mongoose (v8.2.0) for MongoDB object modeling
jsonwebtoken (v9.0.3) for JWT-based authentication
bcryptjs (v3.0.3) for password and PIN hashing
openai (v4.28.0) for AI-based story generation
cors (v2.8.5) for cross-origin request handling
dotenv (v16.4.5) for environment variable management
Development dependencies include jest (v30.2.0) and supertest (v7.2.2) for backend testing, and nodemon (v3.1.0) for development-time auto-restart.

6.9.2 Frontend Dependencies

6.9.3 Frontend Stack

The mobile application is developed using Expo (SDK ∼54.0.33) with React (v19.1.0) and React Native (v0.81.5). Navigation is further managed and controlled by using `expo-router` (∼6.0.23) in conjunction with React Navigation libraries.

Core Frontend Libraries

The primary frontend dependencies include:

- `expo-speech` (∼14.0.8) for on-device text-to-speech functionality.
- `expo-speech-recognition` (v3.1.0) for interactive reading mode.

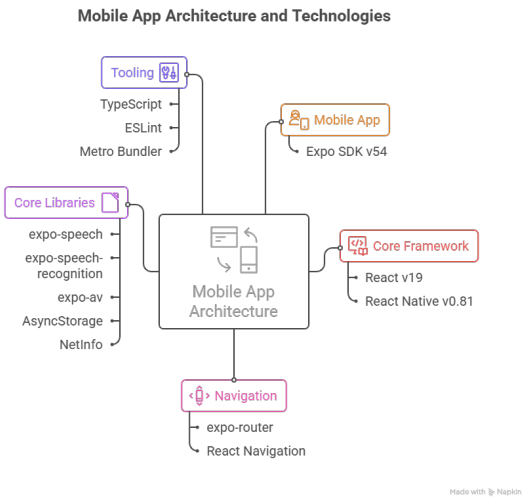

Figure 6.2: Mobile App Architecture and Technologies

- `@react-native-async-storage/async-storage` (v2.2.0) for local data persistence.
- `expo-av` (∼16.0.8) for audio playback and recording.
- `@react-native-community/netinfo` (v11.4.1) for network connectivity monitoring.

Development Tooling

Development tooling includes TypeScript (∼5.9.2) for essentially static typing and ESLint (v9.25.0) with `eslint-config-expo` for code quality enforcement. The Metro bundlers is utilized as part of the Expo toolchain for JavaScript bundling and asset transformation.

6.9.4 Database and AI Integration

The system uses MongoDB as its database, accessed through Mongoose (v8.2.0). AI story generation is implemented using the OpenAI SDK (v4.28.0), while voice narration relies on Expo's native TTS services.

6.9.5 Build and Deployment Tooling

The ToggleTail system is set to follow a standard Node.js development workflow. Package management and task execution are handled using `npm` scripts, which essentially

provide structured commands for starting development servers, installing dependencies, running the backend API, and executing database seed operations.
The system adopts a modulars architecture consisting of:

- A frontend implemented using Expo (React Native framework).
- A backend implemented using Node.js and Express.
- RESTful HTTP communication between frontend and backend during development.

This separation improves maintainability, scalability, and modularity of the overall system design.

6.9.6 Expo Application Services (EAS)

The presence of an `eas.json` configuration file indicates integration with Expo Application Services (EAS) for native mobile builds.
EAS enables:

- Cloud-based iOS and Android build generation.
- Profile-based build configurations.
- Production-ready artifact creation for app store submission.

Multiple build profiles are already defined, typically including development, preview, and production configurations. This further allows the project to remain within the managed mainly within the Expo workflow while supporting native builds without ejecting to a bare React Native setup.

6.9.7 Environment Configuration

Sensitive configuration parameters are managed using environment variables. These include:

- MongoDB connection URI
- OpenAI API credentials
- ElevenLabs API credentials
- Backend service endpoints

This approach ensures secure credential management, environment separation (development vs. production), and improved portability across deployment targets.

6.9.8 Continuous Integration and Deployment Status

At the current stage of development::

- No automated CI/CD pipeline is configured.
- No GitHub Actions workflows are implemented.
- Build and deployment processes are performed manually.

The infrastructures that are set up to therefore supports structured local development and cloud-based mobile builds but does not yet include automated testing or deployment workflows.

Chapter 7

Testing

7.1 Introduction to Testing

tabularx booktabs float
The testing phase was conducted to evaluate whether the ToggleTail application meets its intended design objectives in practice. While the earlier stages of development focused on system architecture and implementation, this phase examines how effectively the application performs when its core features are used as intended. The evaluation focuses on functionality that directly supports the goals of the application, including secure parent authentication, clear separation between parent and child interfaces, reliable story creation and storage, synchronization between highlighted text and narration, and accessible parental controls. In addition to verifying that individual features operate correctly, the testing phase assesses whether the overall experience is understandable, responsive, and appropriate for both children and parents. By linking each test directly to the project objectives, the evaluation ensures that the results provide meaningful insight into the effectiveness and usability of the final application.

7.2 Types of Testing Conducted

In order to evaluate both the technical functionality and the user interaction quality of the ToggleTail application, several complementary testing approaches were used. Each testing type focuses on a different aspect of system evaluation, ensuring that both system behavior and user experience were examined. The following testing categories were applied during the development and evaluation of the application:

- **Unit Testing** Individual logic components of the system were verified to ensure they functioned correctly in isolation. This included validation of password au-

thentication logic, narration playback triggers, story generation functions, and basic navigation handlers within the application.

- **Functional Testing** Functional testing was conducted to verify that the primary features of the application operated according to their intended behavior. These tests included parent authentication, session timeout behavior, story creation and storage, narration playback, and parental control functionality.

- **Usability Testing and User Acceptance** Usability testing was conducted to evaluate how easily both children and parents could interact with the application. Observational sessions were used to assess children's engagement with narrated stories and parents' ability to manage application settings. As for User acceptance testing evaluated whether the application satisfied the intended goals of providing an engaging storytelling experience for children while allowing parents to effectively supervise and configure the reading environment.

  The following sections present the results of these testing approaches and analyze how effectively the ToggleTail system satisfies its functional and usability objectives. However before expanding further first the chapter will identify and explain the testing that was done throughout the development phase, including feedback provided by the projects supervisor, respectively. It is also important to note that order in which the testing phases and results are shown correlates with the chronological order of their execution.

7.3 Design and Implementation Challenges

During the development of the ToggleTail application, several design and implementation challenges were identified prior to and during the testing phase. Addressing these challenges was necessary to ensure that the application met both its functional requirements and usability objectives prior to being presented the Usability and Testing Phase. These adjustments improved system stability, interaction clarity, and overall user experience before the final evaluation stages were conducted.

Challenge Category	Issue Identified	Adjustment Implemented	Outcome
User Interface Design	Initial interface elements were visually cluttered and difficult for young users to interpret.	The layout was simplified by reducing visual elements, enlarging key navigation controls, and clearly separating reading and navigation actions.	Improved usability for children and easier story navigation.
Parent–Child Interface Separation	Children could potentially access parent configuration areas if navigation was unclear.	A dedicated parent authentication gate was implemented to protect configuration settings and ensure role-based access control.	Secure separation between parent controls and child interface.
Story Storage Structure	Generated stories initially produced duplicate metadata entries during local storage.	Storage logic was revised to prevent redundant metadata creation and improve database structure.	More efficient story storage and improved system reliability.
Narration Loading Delay	Narration generation occasionally caused noticeable loading delays during story playback.	Loading feedback indicators were introduced and narration caching was implemented to improve perceived responsiveness.	Reduced user confusion during narration initialization.
Text Highlight Synchronization	Early versions showed slight timing mismatches between narration and highlighted text.	Synchronization logic was refined to align highlighted words with the exact narration timing.	Improved read-along experience supporting literacy engagement.
Session Timeout Behavior	Parent session timeout behavior was inconsistent during early testing stages.	Session monitoring logic was updated to ensure automatic locking after inactivity.	Improved security and session management reliability.

Table 7.1: Implementation Challenges Identified During Development

Addressing these design and implementation challenges ensured that the system reached a stable and usable state prior to the formal evaluation stages. This essentially was done throughout the development process specifically when it came to demonstrating the functionality of the application to the project supervisor informally and being able to receive that feedback while also going in and adjusting it so that the changes are reflected and that the application is presenting its best foot forward when it comes to the usability testing and user acceptance The improvements made during development helped enhance both system reliability and user interaction quality, enabling the subsequent functional and usability testing phases to focus on evaluating the application under realistic usage conditions.

7.4 Functional Testing

Functional testing was conducted to verify that each core feature of the ToggleTail application performed according to its expected behavior and supported the objectives of the system. Testing was performed using the Expo Go environment on a physical mobile device in order to evaluate the application under realistic usage conditions. An automated testing approach was used to simulate typical user interactions and verify whether system responses aligned with the intended functionality.
Functional test cases were designed to evaluate key components of the application,

including parent authentication and session control, story creation and storage, narration playback, synchronization between highlighted text and spoken narration, parental controls, and offline story accessibility. The tests also included validation of error conditions such as incorrect login attempts and narration loading delays. In addition, multiple child profiles with different reading levels were used during testing to ensure that the story generation and reading support features adapted appropriately. Each test case was evaluated by comparing the expected system behavior with the observed outcome, and any inconsistencies were documented along with corrective actions taken to improve system reliability and usability. The table below outline six different test cases alongside the project objectives that it correlates with directly and specifically looks at the expected result the actual result prior to adjusting and then the action taken.

Test Case	Linked Project Objective	Test Description	Expected Result	Actual Result	Action Taken
Parent Login Authentication	Secure parent access and role-based control	Parent attempts to access the Parent View using the correct password	Parent dashboard unlocks only after correct authentication	Authentication succeeded and Parent View opened correctly	No change required
Invalid Parent Login Attempt	Secure parent access and system security	Parent enters incorrect password three times	System denies access and displays authentication error	Error message displayed and Parent View remained locked	Improved clarity of error message
Session Timeout Lock	Secure parent access and session control	Parent remains inactive in Parent View for more than 30 minutes	Parent View locks automatically and requires reauthentication	Session lock triggered correctly after inactivity	Adjusted session timer
Parent Dashboard Navigation	Parent configuration interface usability	Parent navigates betweens story management, narration settings, and screen time controls	All parent configuration features load correctly	All navigation paths functioned correctly	No action required
Story Creation with Child Profile	Story creation and management functionality	Parent creates a stories including child name and reading level	Story generated correctly and stored in the story library	Story generated and stored successfully	Improved metadata structure
Story Storage Efficiency	Efficient local storage architecture	Generated stories and metadata are stored locally without duplicate entries	Story stored with efficient metadata structure	Duplicate metadata fields detected initially	Adjusted storage logic

Table 7.2: Functional Testing Results Linked to ToggleTail Project Objectives

7.5 Objective-Linked Testing

While functional testing verified whether the core features of the ToggleTail system operated correctly, an additional evaluation stage was conducted to ensure that these features directly supported the intended project objectives. This objective-linked test-

ing approach focused on validating whether the implemented systems behavior aligned with the educational, usability, and parental control goals defined during the design phase. By explicitly mapping each testing activities to a predefined objective, the evaluation ensured that the system was not only technically functional but also effective in supporting its intended purpose.

Defined System Objectives

The following objectives were used to guide the objective-linked evaluation:

- **F1: Secure Parent Authentication** – Ensure that parent-only features are protected through authentication and session control mechanisms.
- **F2: Parent–Child Interface Separation** – Maintain a clear separation between the child reading interface and the parent configuration dashboard.
- **F3: Reliable Story Creation and Storage** – Enable parents to generate and store stories efficiently without duplication or data inconsistencies.
- **F4: Narration and Text Synchronization** – Ensure that highlighted text corresponds accurately with the spoken narration during story playback.
- **F5: Accessible Parental Controls** – Provide parents with clear access to configuration features such as screen time limits and narration settings.

Testing Area	Evaluation Focus	Linked Objective(s)	Outcome Summary
Parent Authentication	Verify that the parent dashboard remains protected through password authentication and session timeout mechanisms.	F1	Authentication and session timeout mechanisms functioned correctly.
Interface Navigation Separation	Ensure that children cannot access parent configuration features during normal application use.	F2	Parent dashboard remained inaccessible from the child interface without authentication.
Story Creation and Storage	Evaluate whether generated stories are stored correctly without duplication and are accessible within the story library.	F3	Storage structure improved after resolving duplicate metadata entries.
Narration and Text Highlighting	Observe whether spoken narration aligns with the highlighted words during story playback.	F4	Synchronization improved after refining timing logic.
Parental Control Accessibility	Verify that parents can locate and configure features such as narration settings and screen time limits.	F5	Parents successfully accessed configuration features with minimal navigation difficulty.

Table 7.3: Objective-Linked Evaluation of ToggleTail System Features

Unlike functional testing, which primarily verifiess whether individual features operate correctly, objective-linked testing evaluates whether those features effectively fulfill the

intended design goals of the system. The results indicate that the ToggleTail application successfully supports the majority of the defined objectives, particularly in areas related to authentication, interface separation, and parental configuration. However, minor improvements remain possible in areas suchh as narration responsiveness and discoverability of advanced configuration options.

7.6 Usability Testing and User Acceptance

Usability testing and user acceptance evaluation were conducted to assess how effectively the ToggleTail application supports its intended interaction goals for both children and parents. The testing sessions were performed using the Expo Go enviroments on a physical mobile device to replicate realistic usage conditions. All sessions were conducted in a quiet indoor enviroment in order to simulate a typical home reading setting. Participants were allowed to interact with the system without strict time limitations so that natural interaction patterns could be observed..

Two groups of participants were involved in the evaluation. The first group consisted of three children between the ages of five and seven who interacted with the storytelling interface. The second group consisted of three adult participants whom are parents who tested the parental configuraton interface. Observations focused on usability, interaction clarity, and whether the implemented features supported the intended educational and parental supervision objectives of the application.

7.6.1 Child Observation Results

The child usability sessions were designed to evaluate whether young users could comfortably interact with the storytelling interface and engage with the narrated reading experience. Each participant was asked to select stories from the story library and listen to narration while the synchronized text highlighting feature was active. Observations focused on behaviors that correspond directly to the system objectives, including comfort with the interface, independent navigation of the story library, engagement with narration playback, and the effectiveness of synchronized highlighting in guiding reading attention. Each criterion was recorded using a binary outcome of Yes (Y) when the behavior was successfully demonstrated and No (N) when minor difficulties or inconsistencies were observed.

Criterion	Linked Objective	C1	C2	C3	Observation Notes
Child appears comfortable interacting with the interface	Child-friendly reading experience	N	Y	Y	All participants interacted comfortably with the story interface. Child 3 showed slight hesitation initially but adapted quickly after exploring the story library.
Child successfully opens a story independently	Child usability and intuitive navigation	Y	Y	Y	All participants were able to locate the story library and open stories without assistance after brief exploration.
Child remains attentive during narration playback	Engagement and storytelling immersion	Y	N	Y	Children 1 and 3 remained attentive during narration playback. Child 2 briefly lost attention during narration loading but regained focus once narration began.
Child visually follows highlighted words while narration plays	Literacy support and read-along synchronization	Y	Y	Y	All participants appeared to visually track highlighted words during narration, indicating effective synchronization between narration and text highlighting.
Child understands playback controls (play/pause)	Accessible child interaction controls	Y	Y	N	All children were able to use the play and pause controls correctly after brief exploration of the interface.
Child navigates between stories with minimal errors	Child-friendly interface design	N	Y	Y	Participants successfully returned to the story library and selected additional stories with minimal hesitation.
Child does not accidentally enter parent controls	Secure parent access separation	Y	Y	Y	Parent controls remained inaccessible to all children due to the implemented parent gate security mechanism.
Loading delays do not cause disengagement	Smooth reading experience	Y	N	Y	Minor narration loading delays were observed. Child 2 briefly showed distraction during loading, suggesting that clearer loading indicators could further improve engagement.

The results indicate that the ToggleTail interface successfully supports a child-friendly reading environment. All participants were able to independently locate and open stories, demonstrating that the navigation structure of the application is intuitive for young users. In addition, the synchronized narration, and highlighted text effectively guided visual attention, supporting the literacy objective of encouraging children to follow along during story playback. Minor disengagement occurred during narration loading for one participant, suggesting that clearer feedback during loading periods may further enhance the reading experience. Overall, the observations indicate that the application provides an engaging and accessible storytelling experience for children within the intended age group..

7.6.2 Parent Observation Results

The parent usability sessions were conducted to evaluate whether the configuration interface enables parents to easily manage and supervise the storytelling environment. Participants were asked to perform common administrative tasks including unlocking

the parent dashboard, creating a story using a child profile, modifying narration settings, and adjusting screen time limits. Observations focused on how easily participants could locate and configure these controls and whether the interface structure allowed parents to complete tasks efficiently without guidance. As with the child observations, results were recorded using Yes (Y) when tasks were completed successfully and No (N) when participants experienced minor difficulty.

Criterion	Linked Objective	P1	P2	P3	Observation Notes
Parent successfully unlocks Parent View	Secure parent authentication	Y	Y	Y	All parents were able to unlock the parent dashboard using the correct password without assistance.
Parent locates story creation feature	Story creation and management usability	Y	Y	Y	All participants located the story creation interface within the parent dashboard after brief navigation.
Parent creates a story using child profile	Personalized story generation	Y	Y	Y	Each parent successfully created a story by selecting the child profile and confirming the reading level settings.
Parent modifies narration configuration	Narration customization controls	Y	N	Y	Parents 1 and 3 successfully adjusted narration settings. Parent 2 initially required additional navigation before locating the narration configuration panel.
Parent locates and adjusts screen time controls	Parental control accessibility	Y	Y	Y	All parents were able to find the screen time settings and adjust the reading time limit for the child profile.
Parent navigates between dashboard features efficiently	Parent interface usability	Y	Y	Y	All participants navigated between story management, narration settings, and parental controls without confusion.
Parent logs out or exits parent dashboard securely	Secure session management	Y	Y	Y	All parents were able to exit the parent dashboard and return to the child interface without difficulty.
Parent understands interface structure without guidance	Overall parent usability objective	Y	Y	Y	Participants demonstrated clear understanding of the parent interface structure and required minimal exploration to complete tasks.

The parent evaluation results indicate that the configuration interface provides clear and accessible controls for managing the storytelling environment. All participants were able to authenticate access to the parent dashboard and successfully create personalized stories for the child profile. Screen time configuration and navigation between dashboard sections were also completed without significant difficulty, suggesting that the interface supports efficient parental supervision. One participant experienced brief difficulty locating narration configuration settings, indicating that improved labeling or visual hierarchy could further enhance discoverability of advanced controls. Overall, the usability sessions confirm that the system effectively enables parents to configure and supervise the reading experience while maintaining secure separation between parent and child interfaces.

7.7 Limitations of Testing

Although the testing process provided useful insight into the functionality and usability of the ToggleTail application, several limitations should be acknowleged. The usability testing sessions involved a relatively small number of participants, consisting of three children and three adults acting as parents. While this allowed for observation of general interaction patterns, a larger and more diverse participant group could provide a more comprehensive evaluation of the system's usability and engagement features.
In addition, the testing enviroment was controlled and relatively quiet, which may not fully represent real-world usage conditions where distractions may occur. Performance testing was also limited to a single mobile device using the Expo Go testing enviroment, meaning that behaviour on different devices or operating systems may vary slightly.
Finally, due to time constraints within the development schedule, long-term usage behaviour such as repeated daily interaction and extended story library growth was not extensively evaluated. Future testing could therefore include larger user groups, longer observation periods, and testing across multiple devices in order to provide a more complete assessment of the system's real-world performance and usability.

Chapter 8

Conclusion

8.1 Conclusion Future work

This project presented the design and implementation of ToggleTails, a child centered storytelling application that combines AI generated narration with human recorded narration in one system. The main goal was to address some of the limitations seen in existing storytelling platforms, especially in terms of customization, lack of user controlled voice options, and the need for a more private and offline friendly solution.

The system was able to demonstrate a working dual mode narration pipeline, where both AI and human narration can be used and switched between without affecting the overall experience. It also implements role based access control, which separates the Parent View and Child View, making sure that configuration and sensitive features are only accessible to guardians. In addition, the use of a local first approach allowed the system to store data on the device, which supports privacy and reduces dependence on internet connectivity.

From a technical perspective, the project shows that it is possible to build a modular mobile application that supports narration customization, voice recording, and playback control without making the system overly complex. The prototype serves as a proof of concept rather than a complete product, but it still demonstrates that AI narration can be used alongside human narration in a way that supports, rather than replaces, human interaction.

It is also important to mention that the focus of this project was on system design and implementation, not on large scale testing or experiments. The validation carried out was mainly to check that the system works as intended. Even with this limitation, the project provides a good foundation for future work in AI assisted storytelling systems, especially those designed for children.

There are several areas where this project can be improved or extended in the future.

One possible direction is adding interactive reading support, where the system can listen to a child reading and provide feedback. This would make the application more engaging and useful for learning, rather than just listening to stories.
Another important step would be to conduct proper user studies to understand how children respond to different narration modes. This could help answer questions about whether AI narration is as effective as human narration, or if one is preferred over the other in certain situations.
The system could also be improved by adding adaptive narration, where the AI adjusts things like speed or tone depending on the child's reading level or behaviour. This would require more advanced processing and may introduce some challenges, but it could make the experience more personalized.
From a system point of view, adding cloud features such as syncing across multiple devices could make the application more practical for real world use. However, this would also raise concerns about privacy and data security, which would need to be handled carefully.
Another area for improvement is expanding the story library and possibly including AI generated stories. If this is done, it would be important to make sure the content remains safe and suitable for children, since this is still a known issue with AI generated text.
Finally, improvements in voice quality and expressiveness could make the narration feel more natural. As text to speech technology continues to improve, this is something that can be explored further in later versions of the system.
Overall, while the current version of ToggleTails is limited to a prototype, it opens up several directions for future development and shows that combining AI and human narration in one platform is both possible and useful.

References

[1] X. Zhang, C. Wang, H. Liao, Z. Li, Y. Wang, L. Wang, D. Jia, Y. Chen, X. Li, Z. Chen, and Z. Wu, "Speechjudge: Towards human-level judgment for speech naturalness," 2025, arXiv:2511.07931. [Online]. Available: http://arxiv.org/abs/2511.07931v1

[2] R. R. Manku, Y. Tang, X. Shi, M. Li, and A. Smola, "Emergenttts-eval: Evaluating tts models on complex prosodic, expressiveness, and linguistic challenges using model-as-a-judge," 2025, arXiv:2505.23009. [Online]. Available: http://arxiv.org/abs/2505.23009v1

[3] C. Minixhofer, O. Klejch, and P. Bell, "Ttsds2: Resources and benchmark for evaluating human-quality text to speech systems," 2025, arXiv:2506.19441. [Online]. Available: http://arxiv.org/abs/2506.19441v1

[4] P. S. Varadhan, S. Thomas, S. T. M S, S. Bhooshan, and M. M. Khapra, "The state of tts: A case study with human fooling rates," in *Proceedings of Interspeech 2025*. ISCA, 2025, pp. 2285–2289.

[5] H. Barakat, O. Turk, and C. Demiroglu, "Deep learning-based expressive speech synthesis: a systematic review of approaches, challenges, and resources," *EURASIP Journal on Audio, Speech, and Music Processing*, vol. 2024, no. 1, 2024.

[6] N. Ellinas, M. Christidou, A. Vioni, S. J. Sung, A. Chalamandaris, P. Tsiakoulis, and P. Mastorocostas, "Controllable speech synthesis by learning discrete phoneme-level prosodic representations," *Speech Communication*, vol. 146, pp. 22–31, 2022.

[7] F. Rautenberg, M. Kuhlmann, F. Seebauer, J. Wiechmann, P. Wagner, and R. Haeb-Umbach, "Speech synthesis along perceptual voice quality dimensions," 2025, arXiv:2501.08791. [Online]. Available: http://arxiv.org/abs/2501.08791v1

[8] S. Liu, Y. Guo, X. Chen, and K. Yu, "Storytts: A highly expressive text-to-speech dataset with rich textual expressiveness annotations," 2024, arXiv:2404.14946. [Online]. Available: http://arxiv.org/abs/2404.14946v1

[9] M. Daniluk and A. P. Pietrzak, "Comparative analysis of natural and synthesized polish speech," *International Journal of Electronics and Telecommunications*, pp. 361–366, 2024.

[10] P. Bhandari and H. M. Brennan, "Trustworthiness of children stories generated by large language models," 2023, arXiv:2308.00073. [Online]. Available: http://arxiv.org/abs/2308.00073v1

[11] M. Valentini, J. Weber, J. Salcido, T. Wright, E. Colunga, and K. Kann, "On the automatic generation and simplification of children's stories," 2023, arXiv:2310.18502. [Online]. Available: http://arxiv.org/abs/2310.18502v1

[12] L. Ye, J. Jiang, Y. Liu, Y. Ran, D. Chang, and P. Liu, "Colin: A multimodal human-ai co-creation storytelling system to support children's multi-level narrative skills," in *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. ACM, 2025.